



Exam CKA

Certified Kubernetes Administrator (CKA)

Program

Version: 10.0

[Total Questions: 83]

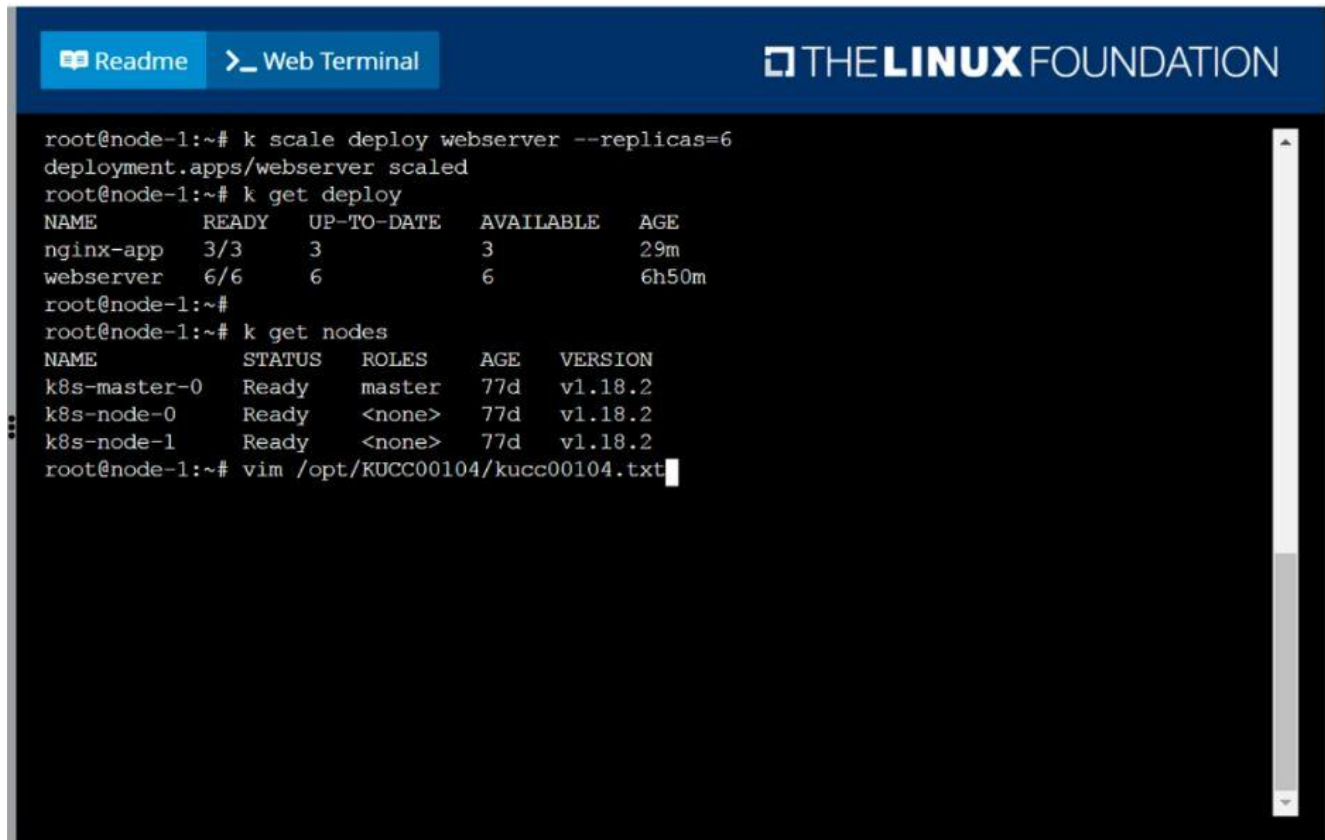
1. SIMULATION

Check to see how many worker nodes are ready (not including nodes tainted NoSchedule) and write the number to /opt/KUCC00104/kucc00104.txt.

Answer:

See the solution below.

Explanation:



The screenshot shows a web terminal interface with a dark background. At the top, there are two tabs: 'Readme' and 'Web Terminal'. The 'Web Terminal' tab is active, and the text 'THE LINUX FOUNDATION' is visible in the top right corner. The terminal shows the following commands and output:

```

root@node-1:~# k scale deploy webserver --replicas=6
deployment.apps/webserver scaled
root@node-1:~# k get deploy
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
nginx-app     3/3     3             3            29m
webserver     6/6     6             6            6h50m
root@node-1:~#
root@node-1:~# k get nodes
NAME           STATUS   ROLES    AGE   VERSION
k8s-master-0   Ready   master   77d   v1.18.2
k8s-node-0     Ready   <none>   77d   v1.18.2
k8s-node-1     Ready   <none>   77d   v1.18.2
root@node-1:~# vim /opt/KUCC00104/kucc00104.txt
  
```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\15 B.JPG



ATION

ATION



ATION



ATION

ATION



ATION



First, identify the broken cluster components and investigate what breaks them. The decommissioned cluster used an external etcd server.

Next, fix the configuration of all broken cluster

Answer:

See the solution below.

Explanation:

Task Summary

SSH into node: cka000059

Cluster was migrated to a new machine

It uses an external etcd server

Identify and fix misconfigured components

Bring the cluster back to a healthy state

Step-by-Step Solution

Step 1: SSH into the correct host

```
ssh cka000059
```

Step 2: Check the cluster status

Run:

```
kubectl get nodes
```

If it fails, the kubelet or kube-apiserver is likely broken.

Check kubelet status:

```
sudo systemctl status kubelet
```

Also, check pod statuses in the control plane:

```
sudo crictl ps -a | grep kube
```

or:

```
docker ps -a | grep kube
```

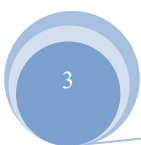
Look especially for failures in kube-apiserver or kube-controller-manager.

Step 3: Inspect the kube-apiserver manifest

Since this is a kubeadm-based cluster, manifests are in:

```
ls /etc/kubernetes/manifests
```

Open kube-apiserver.yaml:





bash

CopyEdit

```
sudo nano /etc/kubernetes/manifests/kube-apiserver.yaml
```

Look for the `--etcd-servers=` flag. If the external etcd endpoint has changed (likely, due to migration), this needs to be fixed.

Example of incorrect configuration:

```
--etcd-servers=https://192.168.1.100:2379
```

If the IP has changed, update it to the correct IP or hostname of the external etcd server.

Also ensure the correct client certificate and key paths are still valid:

```
--etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
```

```
--etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.crt
```

```
--etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key
```

If the files are missing or the path is wrong due to migration, correct those as well.

Step 4: Save and exit, and let static pod restart

Static pod changes will be picked up automatically by the kubelet (watch for `/etc/kubernetes/manifests` changes).

Check again:

```
docker ps | grep kube-apiserver
```

or

```
crictl ps | grep kube-apiserver
```

Step 5: Confirm API is healthy

Once kube-apiserver is up, try:

```
kubectl get componentstatuses
```

```
kubectl get nodes
```

If these commands work and return valid statuses, the control plane is functional again.

Step 6: Check controller-manager and scheduler (optional)

If still broken, check the other static pods in `/etc/kubernetes/manifests/` and correct paths if necessary.

Also verify that `/etc/kubernetes/kubelet.conf` and `/etc/kubernetes/admin.conf` are present and valid.

Command Summary

```
ssh cka000059
```



Check system and kubelet

```
sudo systemctl status kubelet
```

```
docker ps -a | grep kube # or crictl ps -a | grep kube
```

Check manifests

```
ls /etc/kubernetes/manifests
```

```
sudo nano /etc/kubernetes/manifests/kube-apiserver.yaml
```

Fix --etcd-servers and certificate paths if needed

Watch pods restart and confirm:

```
kubectl get nodes
```

```
kubectl get componentstatuses
```

4. SIMULATION

Check the image version in pod without the describe command

Answer:

See the solution below.

Explanation:

```
kubectl get po nginx -o
```

```
jsonpath='{.spec.containers[].image}{"\n"}'
```

5. SIMULATION

Score: 4%



Task



Create a persistent volume with name app-data , of capacity 1Gi and access mode ReadOnlyMany. The type of volume is hostPath and its location is /srv/app-data .

Answer:

See the solution below.

Explanation:

Solution:

```
#vi pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: app-config
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadOnlyMany
  hostPath:
    path: /srv/app-config
#
kubectl create -f pv.yaml
```

6. SIMULATION

Create 2 nginx image pods in which one of them is labelled with env=prod and another one labelled with env=dev and verify the same.

Answer:

See the solution below.

Explanation:

```
kubectl run --generator=run-pod/v1 --image=nginx -- labels=env=prod nginx-prod --dry-run
-o yaml > nginx-prodpod.yaml
Now, edit nginx-prod-pod.yaml file and remove entries like
"creationTimestamp: null" "dnsPolicy: ClusterFirst"
```

vim nginx-prod-pod.yaml

apiVersion: v1

kind: Pod

metadata:

labels:

env: prod

name: nginx-prod

spec:

containers:

- image: nginx

name: nginx-prod

restartPolicy: Always

kubectl create -f nginx-prod-pod.yaml

kubectl run --generator=run-pod/v1 --image=nginx --

labels=env=dev nginx-dev --dry-run -o yaml > nginx-dev-pod.yaml

apiVersion: v1

kind: Pod

metadata:

labels:

env: dev

name: nginx-dev

spec:

containers:

- image: nginx

name: nginx-dev

restartPolicy: Always

kubectl create -f nginx-prod-dev.yaml

Verify :

kubectl get po --show-labels

kubectl get po -l env=prod





kubectl get po -l env=dev

7. SIMULATION

List the nginx pod with custom columns POD_NAME and POD_STATUS

Answer:

See the solution below.

Explanation:

```
kubectl get po -o=custom-columns="POD_NAME:.metadata.name,
POD_STATUS:.status.containerStatuses[].state"
```

8. SIMULATION

Get IP address of the pod – “nginx-dev”

Answer:

See the solution below.

Explanation:

Kubect1 get po -o wide

Using JsonPath

```
kubect1 get pods -o=jsonpath='{range
items[*]}{.metadata.name}{"\t"}{.status.podIP}{"\n"}{end}'
```

9. SIMULATION

Score: 7%



Task

Create a new nginx Ingress resource as follows:

- Name: ping
- Namespace: ing-internal
- Exposing service hi on path /hi using service port 5678

The availability of service hi can be checked using the following command, which should return hi :

```
[student@node-1] $ | curl
-kL <INTERNAL_IP>/hi
```

Answer:

See the solution below.

Explanation:

Solution:

```
vi ingress.yaml
```

```
#
```

```
apiVersion: networking.k8s.io/v1
```

```
kind: Ingress
```

```
metadata:
```

```
name: ping
```

```
namespace: ing-internal
```

```
spec:
```

```
rules:
```

```
- http:
```

```
paths:
```

```
- path: /hi
pathType: Prefix
backend:
service:
name: hi
port:
number: 5678
#
kubectl create -f ingress.yaml
```

10. SIMULATION

Create a busybox pod that runs the command “env” and save the output to “envpod” file

Answer:

See the solution below.

Explanation:

Solution:

```
vi ingress.yaml
#
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ping
namespace: ing-internal
spec:
rules:
- http:
paths:
- path: /hi
pathType: Prefix
backend:
```

service:

name: hi

port:

number: 5678

#

kubectrl create -f ingress.yaml

11. SIMULATION

Create a Kubernetes secret as follows:

☞ Name: super-secret

☞ password: bob

Create a pod named pod-secrets-via-file, using the redis Image, which mounts a secret named super-secret at /secrets.

Create a second pod named pod-secrets-via-env, using the redis Image, which exports password as

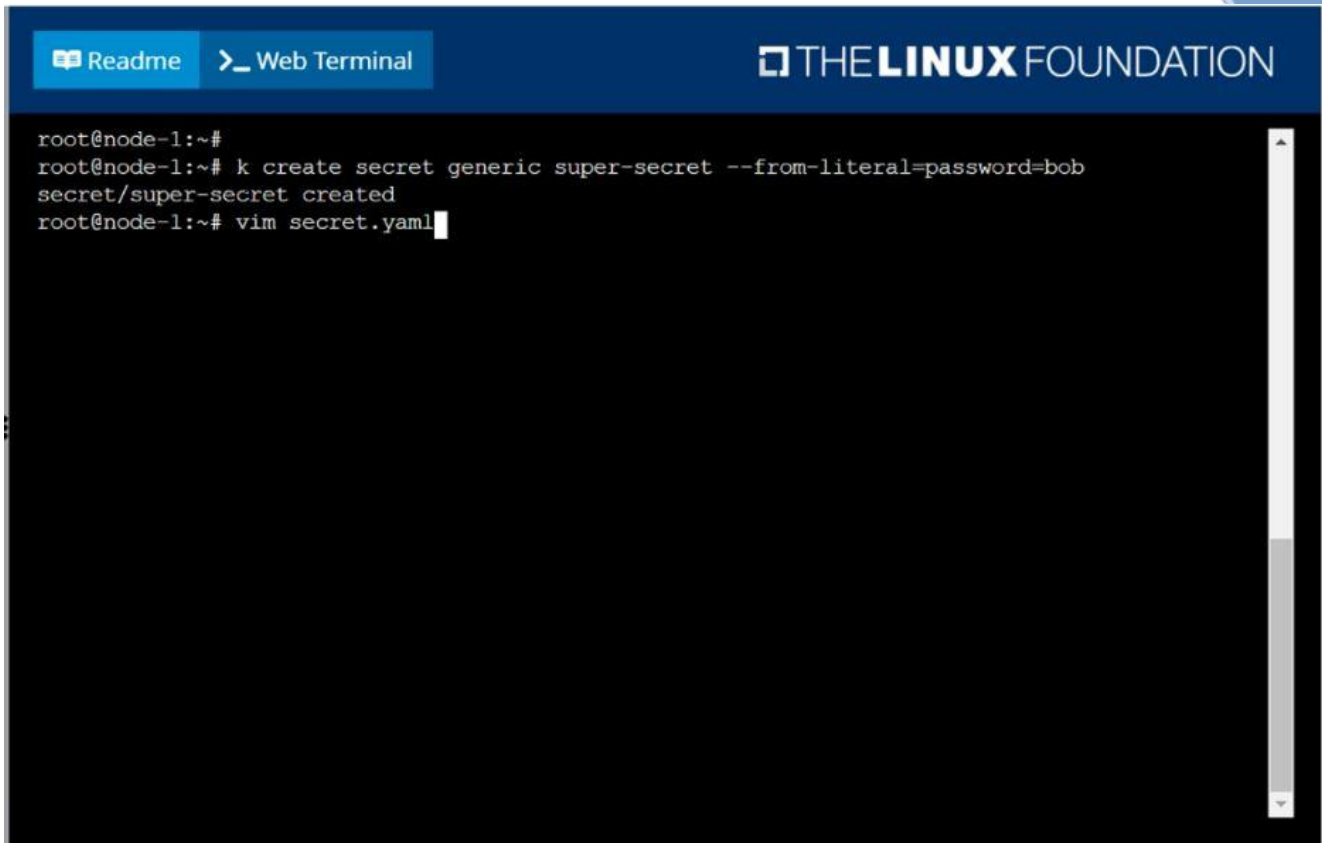
CONFIDENTIAL

Answer:

See the solution below.

Explanation:

solution



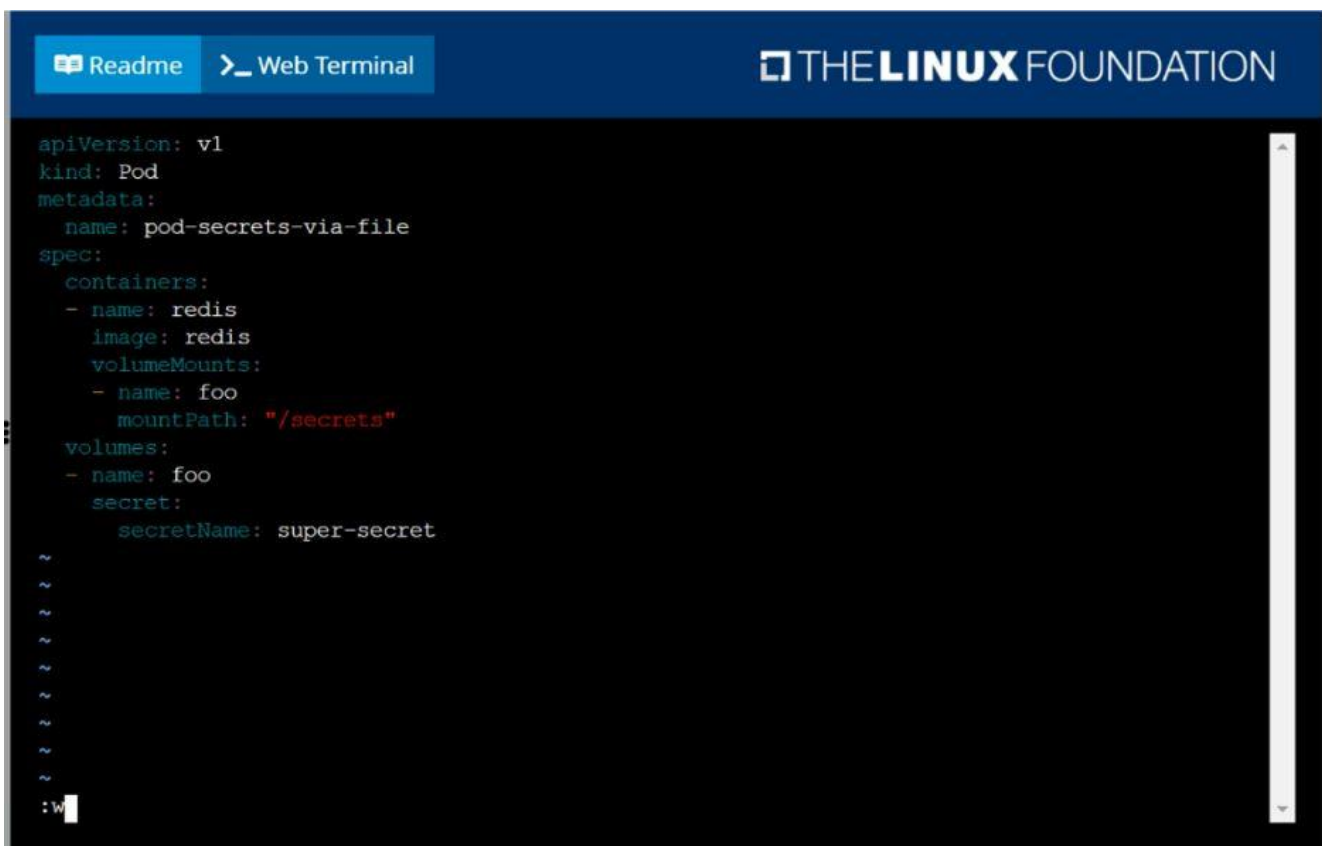
The screenshot shows a web terminal window with a dark background. At the top, there is a blue header bar with a 'Readme' button and a 'Web Terminal' button. To the right of the header is the 'THE LINUX FOUNDATION' logo. The terminal content shows a user at the 'root@node-1' prompt. The user enters the command 'k create secret generic super-secret --from-literal=password=bob', which returns the output 'secret/super-secret created'. The user then enters 'vim secret.yaml'.

```

root@node-1:~#
root@node-1:~# k create secret generic super-secret --from-literal=password=bob
secret/super-secret created
root@node-1:~# vim secret.yaml

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\12 B.JPG



The screenshot shows a web terminal window with a dark background. At the top, there is a blue header bar with a 'Readme' button and a 'Web Terminal' button. To the right of the header is the 'THE LINUX FOUNDATION' logo. The terminal content shows a Kubernetes Pod manifest file. The user enters the command ':w' to save the file.

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-secrets-via-file
spec:
  containers:
  - name: redis
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/secrets"
  volumes:
  - name: foo
    secret:
      secretName: super-secret
~
~
~
~
~
~
~
~
~
:w

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\12 C.JPG

Readme
Web Terminal

```

root@node-1:~# k create -f secret.yaml
pod/pod-secrets-via-file created
root@node-1:~# vim secret1.yaml
root@node-1:~# k create -f secret1.yaml
pod/pod-secrets-via-env created
root@node-1:~# k get po
NAME                                READY   STATUS    RESTARTS   AGE
cpu-utilizer-98b9se                 1/1     Running   0           6h25m
cpu-utilizer-ab2d3s                 1/1     Running   0           6h25m
cpu-utilizer-kipb9a                 1/1     Running   0           6h25m
ds-kusc0020l-2r2k9                  1/1     Running   0           40m
ds-kusc0020l-hzm9q                  1/1     Running   0           40m
foo                                  1/1     Running   0           6h28m
front-end                           1/1     Running   0           6h27m
hungry-bear                         1/1     Running   0           36m
kucc8                                3/3     Running   0           34m
nginx-app-848cfcf495-9prjh          1/1     Running   0           19m
nginx-app-848cfcf495-gl2kh          1/1     Running   0           19m
nginx-app-848cfcf495-pg2c8          1/1     Running   0           19m
nginx-kusc0010l                     1/1     Running   0           26m
pod-secrets-via-env                 1/1     Running   0           4s
pod-secrets-via-file                1/1     Running   0           106s
webserver-84c55967f4-qzjcv          1/1     Running   0           6h43m
webserver-84c55967f4-t479l          1/1     Running   0           6h43m
root@node-1:~#

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\12 D.JPG

12. SIMULATION

List all the pods sorted by name

Answer:

See the solution below.

Explanation:

kubectl get pods --sort-by=.metadata.name

13. SIMULATION

List pod logs named "frontend" and search for the pattern "started" and write it to a file "/opt/error-logs"

Answer:

See the solution below.

Explanation:

Kubectl logs frontend | grep -i "started" > /opt/error-logs



14. SIMULATION

You must connect to the correct host. Failure to do so may result in a zero score.

```
[candidate@base] $ ssh Cka000037
```

Context

A legacy app needs to be integrated into the Kubernetes built-in logging architecture (i.e. kubectl logs).

Adding a streaming co-located container is a good and common way to accomplish this requirement.

Task

Update the existing Deployment synergy-leverager, adding a co-located container named sidecar using the busybox:stable image to the existing Pod . The new co-located container has to run the following command:

```
/bin/sh -c "tail -n+1 -f /var/log/synergy-leverager.log"
```

Use a Volume mounted at /var/log to make the log file synergy-leverager.log available to the co-located container .

Do not modify the specification of the existing container other than adding the required volume mount .

Failure to do so may result in a reduced score.

Answer:

See the solution below.

Explanation:

MISSING

15. SIMULATION

Create a pod with image nginx called nginx and allow traffic on port 80

Answer:

See the solution below.

Explanation:

```
kubectl run nginx --image=nginx --restart=Never --port=80
```

16. SIMULATION

Schedule a Pod as follows:

. Name: kucc1



. App Containers : 2

. Container Name/Images :

☞ redis

☞ Memcached

Answer:

See the solution below.

Explanation:

MISSING

17. SIMULATION

Score: 4%



Context

You have been asked to create a new ClusterRole for a deployment pipeline and bind it to a specific ServiceAccount scoped to a specific namespace.

Task

Create a new ClusterRole named deployment-clusterrole, which only allows to create the following resource types:

- Deployment
- StatefulSet
- DaemonSet

Create a new ServiceAccount named cicd-token in the existing namespace app-team1. Bind the new ClusterRole deployment-clusterrole to the new ServiceAccount cicd-token ,



limited to the namespace app-team1.

Answer:

See the solution below.

Explanation:

Solution:

Task should be complete on node k8s -1 master, 2 worker for this connect use command

```
[student@node-1] > ssh k8s
```

```
kubectl create clusterrole deployment-clusterrole --verb=create --
```

```
resource=deployments,statefulsets,daemonsets
```

```
kubectl create serviceaccount cicd-token --namespace=app-team1
```

```
kubectl create rolebinding deployment-clusterrole --clusterrole=deployment-clusterrole --  
serviceaccount=default:cicd-token --namespace=app-team1
```

18. SIMULATION

You must connect to the correct host. Failure to do so may result in a zero score.

```
[candidate@base] $ ssh Cka000022
```

Task

Reconfigure the existing Deployment front-end in namespace spline-reticulator to expose port 80/tcp of the existing container nginx .

Create a new Service named front-end-svc exposing the container port 80/tcp . Configure the new Service to also expose the individual Pods via a NodePort .

Answer:

See the solution below.

Task Summary

SSH into cka000022

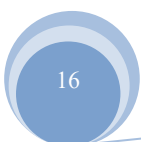
Modify an existing Deployment:

Namespace: spline-reticulator

Deployment: front-end

Container: nginx

Expose: port 80/tcp





Create a Service:

Name: front-end-svc

Type: NodePort

Port: 80 → container port 80

Step-by-Step Solution

* 1. SSH into the correct node

ssh cka000022

Skipping this = zero score

* 2. Edit the Deployment to expose port 80

kubectl edit deployment front-end -n spline-reticulator

Under containers: → nginx, add this if not present:

ports:

- containerPort: 80

protocol: TCP

This enables the container to accept traffic on port 80.

* 3. Create a NodePort Service

Create a file named front-end-svc.yaml:

cat < front-end-svc.yaml

apiVersion: v1

kind: Service

metadata:

name: front-end-svc

namespace: spline-reticulator

spec:

type: NodePort

selector:

app: front-end

ports:

- port: 80

targetPort: 80



protocol: TCP

EOF

Make sure the Deployment has a matching label selector like app: front-end. You can verify with:

```
kubectl get deployment front-end -n spline-reticulator -o yaml | grep labels -A 2
```

* 4. Apply the service

```
kubectl apply -f front-end-svc.yaml
```

* 5. Verify

Check if the service is created and has a NodePort assigned:

```
kubectl get svc front-end-svc -n spline-reticulator
```

You should see something like:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
front-end-svc	NodePort	10.96.0.123	80:3XXXX/TCP	10s	

Where 3XXXX is your automatically assigned NodePort (between 30000–32767).

Final Command Summary

```
ssh cka000022
```

```
kubectl edit deployment front-end -n spline-reticulator
```

Add:

ports:

- containerPort: 80

```
cat < front-end-svc.yaml
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
name: front-end-svc
```

```
namespace: spline-reticulator
```

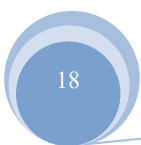
```
spec:
```

```
type: NodePort
```

```
selector:
```

```
app: front-end
```

```
ports:
```



- port: 80

targetPort: 80

protocol: TCP

EOF

kubectl apply -f front-end-svc.yaml

kubectl get svc front-end-svc -n spline-reticulator

19. SIMULATION

From the pod label name=cpu-utilizer, find pods running high CPU workloads and

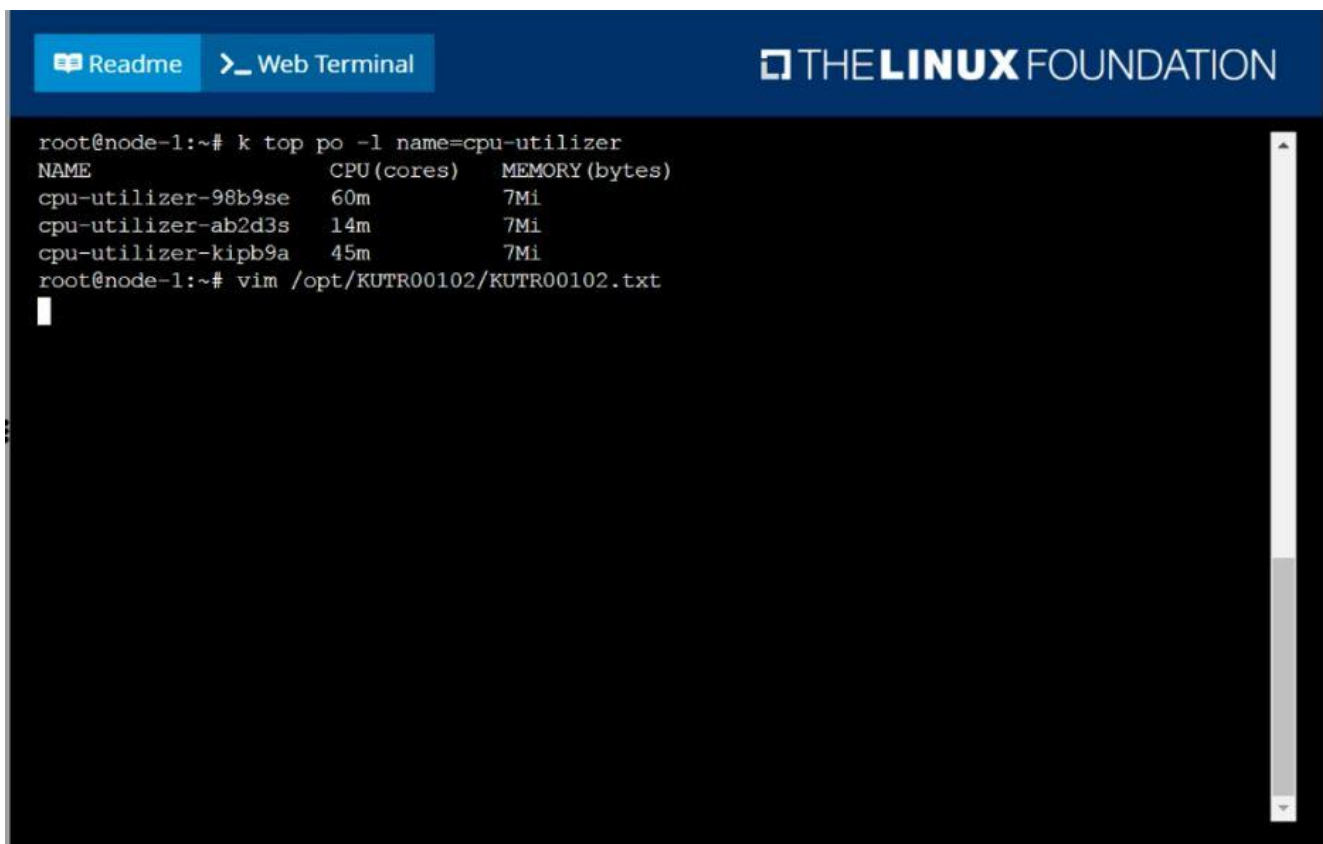
write the name of the pod consuming most CPU to the file /opt/KUTR00102/KUTR00102.txt (which already exists).

Answer:

See the solution below.

Explanation:

solution



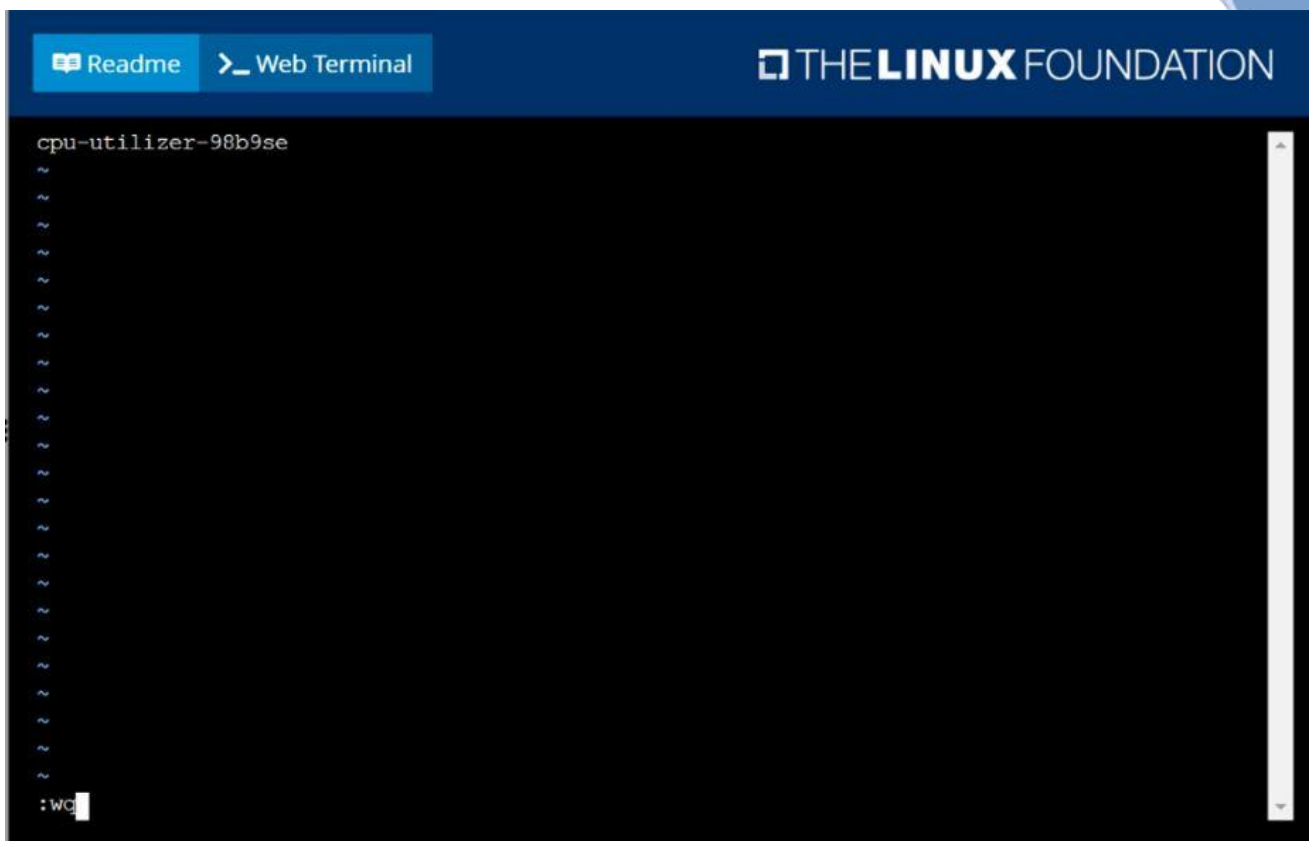
The screenshot shows a web terminal interface with a dark background. At the top, there are two tabs: 'Readme' and 'Web Terminal'. The 'Web Terminal' tab is active. In the top right corner, the 'THE LINUX FOUNDATION' logo is visible. The terminal content shows a user at 'root@node-1' running the command 'k top po -l name=cpu-utilizer'. The output is a table with three columns: 'NAME', 'CPU (cores)', and 'MEMORY (bytes)'. The table lists three pods: 'cpu-utilizer-98b9se' (60m CPU, 7Mi memory), 'cpu-utilizer-ab2d3s' (14m CPU, 7Mi memory), and 'cpu-utilizer-kipb9a' (45m CPU, 7Mi memory). Below the table, the user runs 'vim /opt/KUTR00102/KUTR00102.txt', and the terminal shows a cursor at the start of a new line in the file.

```

root@node-1:~# k top po -l name=cpu-utilizer
NAME                CPU (cores)  MEMORY (bytes)
cpu-utilizer-98b9se  60m          7Mi
cpu-utilizer-ab2d3s  14m          7Mi
cpu-utilizer-kipb9a  45m          7Mi
root@node-1:~# vim /opt/KUTR00102/KUTR00102.txt

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\16 B.JPG



F:\Work\Data Entry Work\Data Entry\20200827\CKA\16 C.JPG



20. SIMULATION

You must connect to the correct host. Failure to do so may result in a zero score.

```
[candidate@base] $ ssh Cka000054
```

Context:

Your cluster 's CNI has failed a security audit. It has been removed. You must install a new CNI that can enforce network policies.

Task

Install and set up a Container Network Interface (CNI) that meets these requirements:

Pick and install one of these CNI options:

- Flannel version 0.26.1 Manifest:

<https://github.com/flannel-io/flannel/releases/download/v0.26.1/kube-flannel.yml>

- Calico version 3.28.2 Manifest:

<https://raw.githubusercontent.com/project-calico/calico/v3.28.2/manifests/tigera-operator.yaml>

Answer:

See the solution below.

Explanation:

Task Summary

SSH into cka000054

Install a CNI plugin that supports NetworkPolicies

Two CNI options provided:

Flannel v0.26.1 (# does NOT support NetworkPolicies)

Calico v3.28.2 #(does support NetworkPolicies)

Decision Point: Which CNI to choose?

#Choose Calico, because only Calico supports enforcing NetworkPolicies natively. Flannel does not.

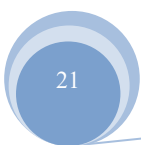
Step-by-Step Solution

1#SSH into the correct node

```
ssh cka000054
```

#Required. Skipping this results in zero score.

2#Install Calico CNI (v3.28.2)





Use the official manifest provided:

```
kubectl apply -f
```

```
https://raw.githubusercontent.com/projectcalico/calico/v3.28.2/manifests/tigera-operator.yaml
```

This installs the Calico Operator, which then deploys the full Calico CNI stack.

3#Wait for Calico components to come up

Check the pods in tigera-operator and calico-system namespaces:

```
kubectl get pods -n tigera-operator
```

```
kubectl get pods -n calico-system
```

#You should see pods like:

```
calico-kube-controllers
```

```
calico-node
```

```
calico-typha
```

```
tigera-operator
```

Wait for all to be in Running state.

4# (Optional) Confirm CNI is enforcing NetworkPolicies

You can check:

```
kubectl get crds | grep networkpolicy
```

You should see:

```
networkpolicies.crd.projectcalico.org
```

This confirms Calico's CRDs are installed for policy enforcement.

Final Command Summary

```
ssh cka000054
```

```
kubectl apply -f
```

```
https://raw.githubusercontent.com/projectcalico/calico/v3.28.2/manifests/tigera-operator.yaml
```

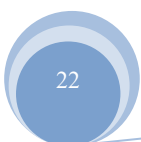
```
kubectl get pods -n tigera-operator
```

```
kubectl get pods -n calico-system
```

```
kubectl get crds | grep networkpolicy
```

21. SIMULATION

List all the pods showing name and namespace with a json path expression





Answer:

See the solution below.

Explanation:

```
kubectl get pods -o=jsonpath="{.items[*]['metadata.name',  
'metadata.namespace']}"
```

22. SIMULATION

You must connect to the correct host.

Failure to do so may result in a zero score. [candidate@base] \$ ssh Cka000046

Task

First, create a new StorageClass named local-path for an existing provisioner named rancher.io/local-path .

Set the volume binding mode to WaitForFirstConsumer .

Not setting the volume binding mode or setting it to anything other than WaitForFirstConsumer may result in a reduced score.

Next, configure the StorageClass local-path as the default StorageClass .

Answer:

See the solution below.

Explanation:

Task Summary

You need to:

SSH into cka000046

Create a StorageClass named local-path using the provisioner rancher.io/local-path

Set the volume binding mode to WaitForFirstConsumer

Make this StorageClass the default

Step-by-Step Solution

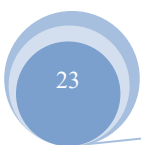
1#SSH into the correct host

```
ssh cka000046
```

#Required. Skipping this = zero score

2#Create a StorageClass YAML file

Create a file named local-path-sc.yaml:





```
cat < local-path-sc.yaml
```

```
apiVersion: storage.k8s.io/v1
```

```
kind: StorageClass
```

```
metadata:
```

```
name: local-path
```

```
annotations:
```

```
storageclass.kubernetes.io/is-default-class: "true"
```

```
provisioner: rancher.io/local-path
```

```
volumeBindingMode: WaitForFirstConsumer
```

```
EOF
```

```
#This:
```

Sets WaitForFirstConsumer (as required)

Marks the class as default using the correct annotation

3# Apply the StorageClass

```
kubectl apply -f local-path-sc.yaml
```

4# Verify it's the default StorageClass

```
kubectl get storageclass
```

You should see local-path with a (default) marker:

```
NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE ALLOWVOLUMEEXPANSION AGE
```

```
local-path rancher.io/local-path Delete WaitForFirstConsumer false 10s
```

Final Command Summary

```
ssh cka000046
```

```
cat < local-path-sc.yaml
```

```
apiVersion: storage.k8s.io/v1
```

```
kind: StorageClass
```

```
metadata:
```

```
name: local-path
```

```
annotations:
```

```
storageclass.kubernetes.io/is-default-class: "true"
```

```
provisioner: rancher.io/local-path
```



volumeBindingMode: WaitForFirstConsumer

EOF

kubectl apply -f local-path-sc.yaml

kubectl get storageclass

23. SIMULATION

Create a nginx pod with label env=test in engineering namespace

Answer:

See the solution below.

Explanation:

kubectl run nginx --image=nginx --restart=Never --labels=env=test

-- namespace=engineering --dry-run -o yaml > nginx-pod.yaml

kubectl run nginx --image=nginx --restart=Never --labels=env=test --

namespace=engineering --dry-run -o yaml | kubectl create -n engineering -f -

YAML File:

apiVersion: v1

kind: Pod

metadata:

name: nginx

namespace: engineering

labels:

env: test

spec:

containers:

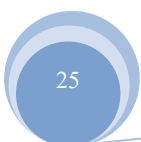
- name: nginx

image: nginx

imagePullPolicy: IfNotPresent

restartPolicy: Never

kubectl create -f nginx-pod.yaml





24. SIMULATION

Quick Reference

HorizontalPodAutoscaler Walkthrough

Documentation , Horizontal Pod Autoscaling , Deployment

You must connect to the correct host .

Failure to do so may result in a zero score.

```
[candidate@base] $ ssh cka000050
```

Task

Create a new HorizontalPodAutoscaler (HPA) named apache-server in the autoscale namespace. This HPA must target the existing Deployment called apache-server in the autoscale namespace.

Set the HPA to aim for 50% CPU usage per Pod . Configure it to have at least 1 Pod and no more than 4 Pods . Also, set the downscale stabilization window to 30 seconds.

Answer:

See the solution below.

Explanation:

MISSING

25. SIMULATION

Get list of all the pods showing name and namespace with a jsonpath expression.

Answer:

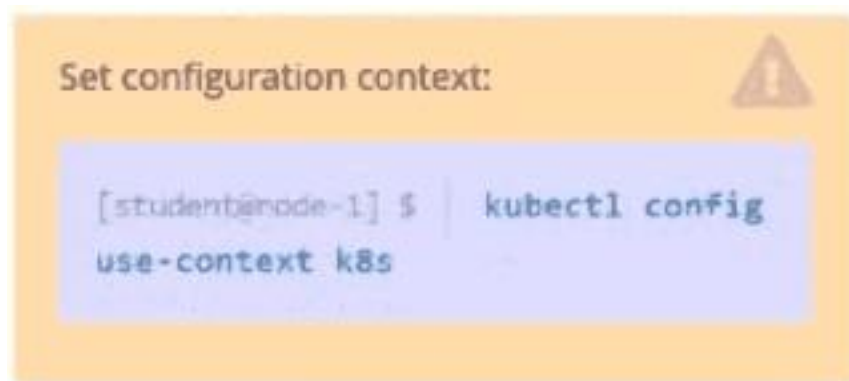
See the solution below.

Explanation:

```
kubectl get pods -o=jsonpath="{.items[*]['metadata.name']  
, 'metadata.namespace']}"
```

26. SIMULATION

Task Weight: 4%



Task

Scale the deployment webserver to 3 pods.

Answer:

See the solution below.

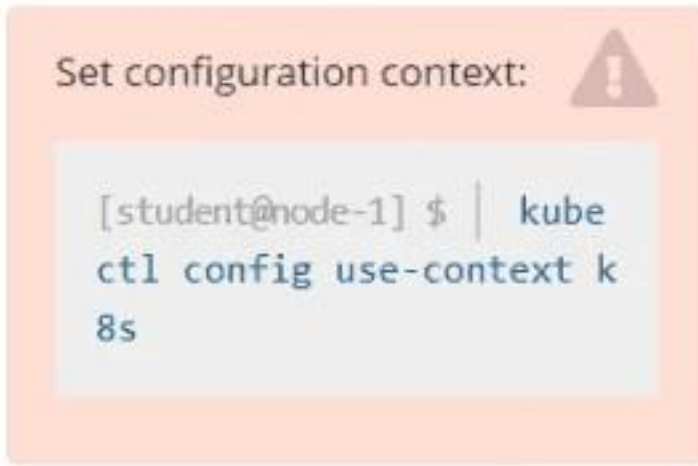
Explanation:

Solution:

```
student@node-1:~$ kubectl scale deploy webserver --replicas=3  
deployment.apps/webserver scaled  
student@node-1:~$ kubectl scale deploy webserver --replicas=3
```

27. SIMULATION

Score: 5%



Task

Monitor the logs of pod bar and:

- Extract log lines corresponding to error file-not-found
- Write them to /opt/KUTR00101/bar

Answer:

See the solution below.

Explanation:

Solution:

```
kubectll logs bar | grep 'unable-to-access-website' > /opt/KUTR00101/bar
```

```
cat /opt/KUTR00101/bar
```

28. SIMULATION

You must connect to the correct host. Failure to do so may result in a zero score.

```
[candidate@base] $ ssh Cka000049
```

Task

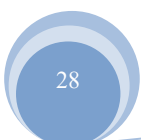
Perform the following tasks:

Create a new PriorityClass named high-priority for user-workloads with a value that is one less than the highest existing user-defined priority class value.

Patch the existing Deployment busybox-logger running in the priority namespace to use the high-priority priority class.

Answer:

See the solution below.





Task Summary

SSH into the correct node: cka000049

Find the highest existing user-defined PriorityClass

Create a new PriorityClass high-priority with a value one less

Patch Deployment busybox-logger (in namespace priority) to use this new PriorityClass

Step-by-Step Solution

1# SSH into the correct node

bash

CopyEdit

ssh cka000049

#Skipping this = zero score

2#Find the highest existing user-defined PriorityClass

Run:

bash

CopyEdit

kubectl get priorityclasses.scheduling.k8s.io

Example output:

vbnet

CopyEdit

NAME VALUE GLOBALDEFAULT AGE

default-low 1000 false 10d

mid-tier 2000 false 7d

critical-pods 1000000 true 30d

Exclude system-defined classes like system-* and the default global one (e.g., critical-pods).

Let's assume the highest user-defined value is 2000.

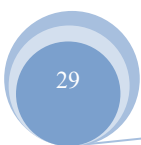
So your new class should be:

Value = 1999

3#Create the high-priority PriorityClass

Create a file called high-priority.yaml:

cat < high-priority.yaml





apiVersion: scheduling.k8s.io/v1

kind: PriorityClass

metadata:

name: high-priority

value: 1999

globalDefault: false

description: "High priority class for user workloads"

EOF

Apply it:

kubectl apply -f high-priority.yaml

4#Patch the busybox-logger deployment

Now patch the existing Deployment in the priority namespace:

kubectl patch deployment busybox-logger -n priority \

--type='merge' \

-p '{"spec": {"template": {"spec": {"priorityClassName": "high-priority"}}}}'

5#Verify your work

Confirm the patch was applied:

kubectl get deployment busybox-logger -n priority -o jsonpath='{.spec.template.spec.priorityClassName}'

#You should see:

high-priority

Also, confirm the class exists:

kubectl get priorityclass high-priority

Final Command Summary

ssh cka000049

kubectl get priorityclass

Create the new PriorityClass

cat < high-priority.yaml

apiVersion: scheduling.k8s.io/v1

kind: PriorityClass

metadata:



name: high-priority

value: 1999

globalDefault: false

description: "High priority class for user workloads"

EOF

kubectl apply -f high-priority.yaml

Patch the deployment

kubectl patch deployment busybox-logger -n priority \

--type='merge' \

-p '{"spec": {"template": {"spec": {"priorityClassName": "high-priority"}}}}'

Verify

kubectl get deployment busybox-logger -n priority -o jsonpath='{.spec.template.spec.priorityClassName}'

kubectl get priorityclass high-priority

29. SIMULATION

Score: 4%



Task

Scale the deployment presentation to 6 pods.

Answer:

See the solution below.

Explanation:

Solution:



```
kubectl get deployment
```

```
kubectl scale deployment.apps/presentation --replicas=6
```

30. SIMULATION

Print pod name and start time to “/opt/pod-status” file

Answer:

See the solution below.

Explanation:

```
kubect1 get pods -o=jsonpath='{range  
items[*]}{.metadata.name}{"\t"}{.status.podIP}{"\n"}{end}'
```

31. SIMULATION

You must connect to the correct host. Failure to do so may result in a zero score.

```
[candidate@base] $ ssh Cka000055
```

Task

Verify the cert-manager application which has been deployed to your cluster .

Using kubectl, create a list of all cert-manager Custom Resource Definitions (CRDs) and save it to ~/resources.yaml .

You must use kubectl 's default output format. Do not set an output format.

Failure to do so will result in a reduced score.

Using kubectl, extract the documentation for the subject specification field of the Certificate Custom Resource and save it to ~/subject.yaml.

Answer:

See the solution below.

Explanation:

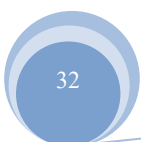
Task Summary

You need to:

SSH into the correct node: cka000055

Use kubectl to list all cert-manager CRDs, and save that list to ~/resources.yaml

Do not use any output format flags like -o yaml





Extract the documentation for the spec.subject field of the Certificate custom resource and save it to
~/subject.yaml

Step-by-Step Instructions

Step 1: SSH into the node

```
ssh cka000055
```

Step 2: List cert-manager CRDs and save to a file

First, identify all cert-manager CRDs:

```
kubectl get crds | grep cert-manager
```

Then extract them without specifying an output format:

```
kubectl get crds | grep cert-manager | awk '{print $1}' | xargs kubectl get crd > ~/resources.yaml
```

This saves the default kubectl get output to the required file without formatting flags.

Step 3: Get documentation for spec.subject in the Certificate CRD

Run the following command:

```
kubectl explain certificate.spec.subject > ~/subject.yaml
```

This extracts the field documentation and saves it to the specified file.

If you're not sure of the resource, verify it exists:

```
kubectl get crd certificates.cert-manager.io
```

Final Command Summary

```
ssh cka000055
```

```
kubectl get crds | grep cert-manager | awk '{print $1}' | xargs kubectl get crd > ~/resources.yaml
```

```
kubectl explain certificate.spec.subject > ~/subject.yaml
```

32. SIMULATION

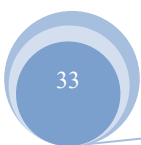
Given a partially-functioning Kubernetes cluster, identify symptoms of failure on the cluster.

Determine the node, the failing service, and take actions to bring up the failed service and restore the health of the cluster. Ensure that any changes are made permanently.

You can ssh to the relevant nodes (bk8s-master-0 or bk8s-node-0) using: [student@node-1] \$ ssh <nodename>

You can assume elevated privileges on any node in the cluster with the following command:

```
[student@nodename] $ | sudo -i
```





Answer:

See the solution below.

Explanation:

solution

```

Readme  Web Terminal  THE LINUX FOUNDATION

root@node-1:~#
root@node-1:~# kubectl config use-context bk8s
Switched to context "bk8s".
root@node-1:~# ssh bk8s-master-0
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.4.0-1109-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Are you ready for Kubernetes 1.19? It's nearly here! Try RC3 with
   sudo snap install microk8s --channel=1.19/candidate --classic
   https://microk8s.io/ has docs and details.

4 packages can be updated.
1 update is a security update.

New release '18.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

student@bk8s-master-0:~$ sudo -i
root@bk8s-master-0:~# vim /var/lib/kubelet/config.yaml

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\23 C.JPG

Readme
Web Terminal

```

authorization:
  mode: Webhook
  webhook:
    cacheAuthorizedTTL: 0s
    cacheUnauthorizedTTL: 0s
clusterDNS:
- 10.96.0.10
clusterDomain: cluster.local
cpuManagerReconcilePeriod: 0s
evictionPressureTransitionPeriod: 0s
fileCheckFrequency: 0s
healthzBindAddress: 127.0.0.1
healthzPort: 10248
httpCheckFrequency: 0s
imageMinimumGCAge: 0s
kind: KubeletConfiguration
nodeStatusReportFrequency: 0s
nodeStatusUpdateFrequency: 0s
rotateCertificates: true
runtimeRequestTimeout: 0s
staticPodPath: /etc/kubernetes/manifests
streamingConnectionIdleTimeout: 0s
syncFrequency: 0s
volumeStatsAggPeriod: 0s
:wc

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\23 D.JPG

Readme
Web Terminal

```

https://microk8s.io/ has docs and details.

4 packages can be updated.
1 update is a security update.

New release '18.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

student@bk8s-master-0:~$ sudo -i
root@bk8s-master-0:~# vim /var/lib/kubelet/config.yaml
root@bk8s-master-0:~# systemctl restart kubelet
root@bk8s-master-0:~# systemctl enable kubelet
root@bk8s-master-0:~# kubectl get nodes

NAME             STATUS    ROLES    AGE   VERSION
bk8s-master-0    Ready     master   77d   v1.18.2
bk8s-node-0      Ready     <none>   77d   v1.18.2
root@bk8s-master-0:~#
root@bk8s-master-0:~# exit
logout
student@bk8s-master-0:~$ exit
logout
Connection to 10.250.4.77 closed.
root@node-1:~#

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\23 E.JPG



33. SIMULATION

Check the Image version of nginx-dev pod using jsonpath

Answer:

See the solution below.

Explanation:

kubect1 get po nginx-dev -o

jsonpath='{.spec.containers[].image}{"\n"}'

34. SIMULATION

Score: 4%



Task

Schedule a pod as follows:

- Name: nginx-kusc00401
- Image: nginx
- Node selector: disk=ssd

Answer:

See the solution below.

Explanation:

Solution:

#yaml

apiVersion: v1

```
kind: Pod
metadata:
name: nginx-kusc00401
spec:
containers:
- name: nginx
image: nginx
imagePullPolicy: IfNotPresent
nodeSelector:
disk: spinning
#
kubectl create -f node-select.yaml
```

35. SIMULATION

Create a pod that echo “hello world” and then exists. Have the pod deleted automatically when it’s completed

Answer:

See the solution below.

Explanation:

```
kubectl run busybox --image=busybox -it --rm --restart=Never --
```

```
/bin/sh -c 'echo hello world'
```

```
kubectl get po # You shouldn't see pod with the name "busybox"
```

36. SIMULATION

Perform the following tasks:

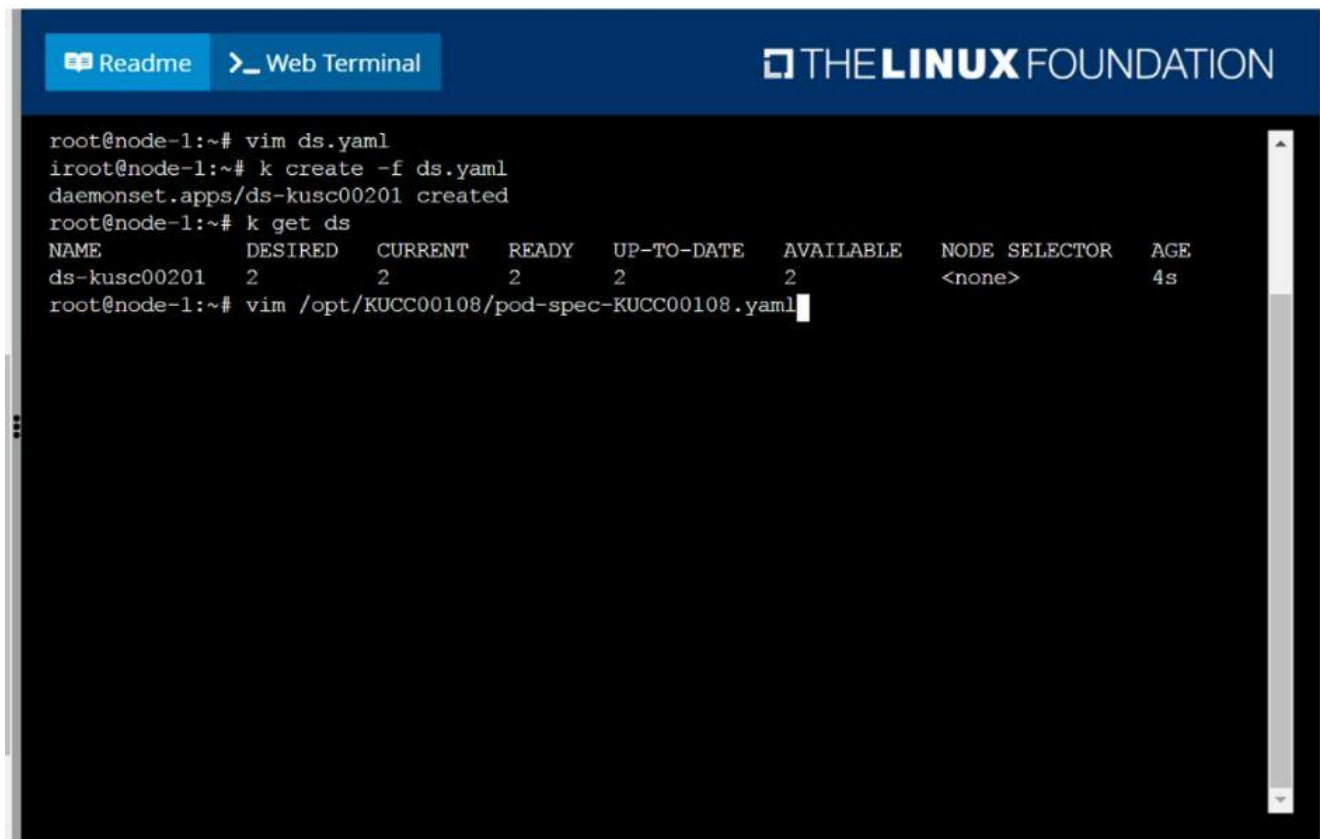
- ☞ Add an init container to hungry-bear (which has been defined in spec file /opt/KUCC00108/pod-spec-KUCC00108.yaml)
- ☞ The init container should create an empty file named /workdir/calm.txt
- ☞ If /workdir/calm.txt is not detected, the pod should exit
- ☞ Once the spec file has been updated with the init container definition, the pod should be created

Answer:

See the solution below.

Explanation:

solution

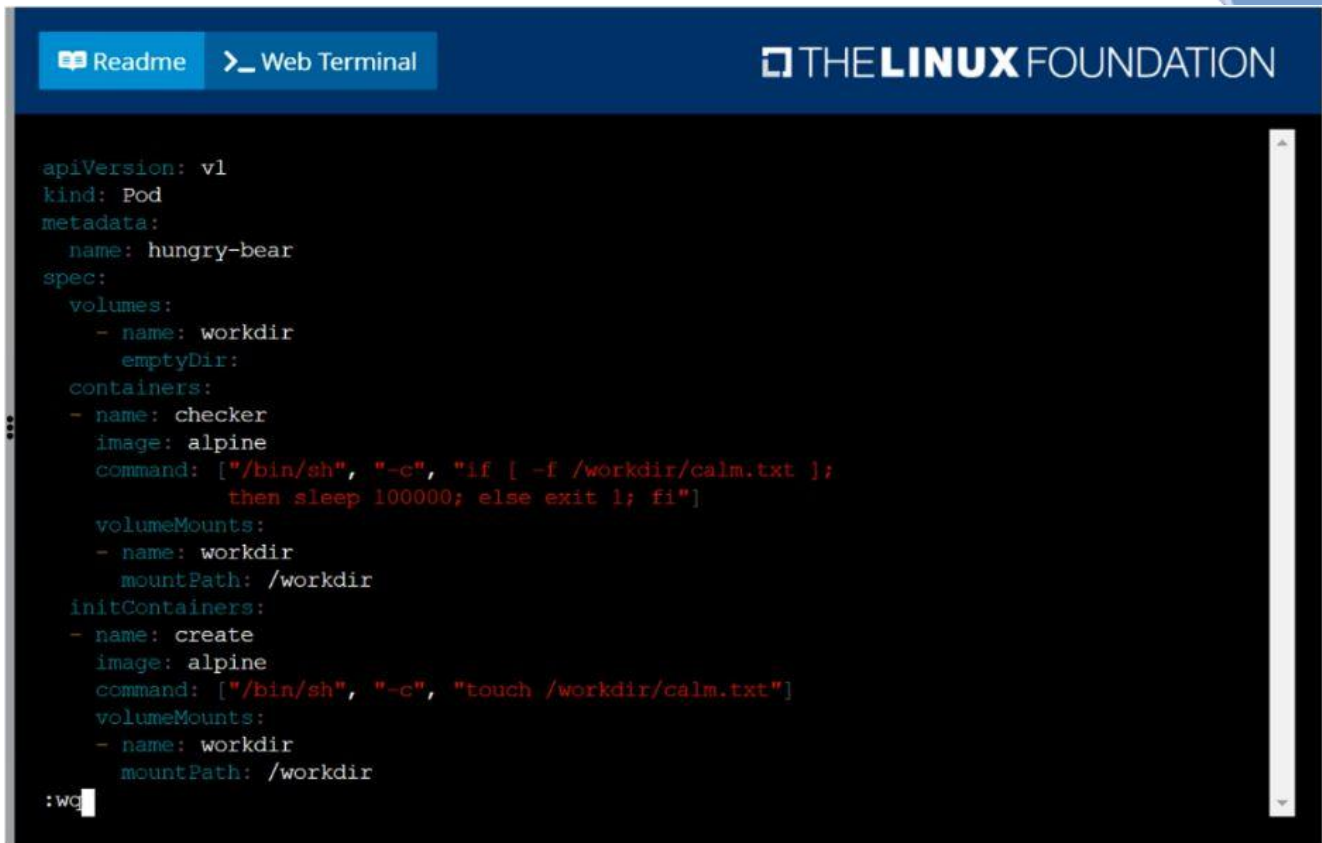


```

root@node-1:~# vim ds.yaml
iroot@node-1:~# k create -f ds.yaml
daemonset.apps/ds-kusc00201 created
root@node-1:~# k get ds
NAME                DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
ds-kusc00201        2         2         2       2            2           <none>          4s
root@node-1:~# vim /opt/KUCC00108/pod-spec-KUCC00108.yaml

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\4 B.JPG

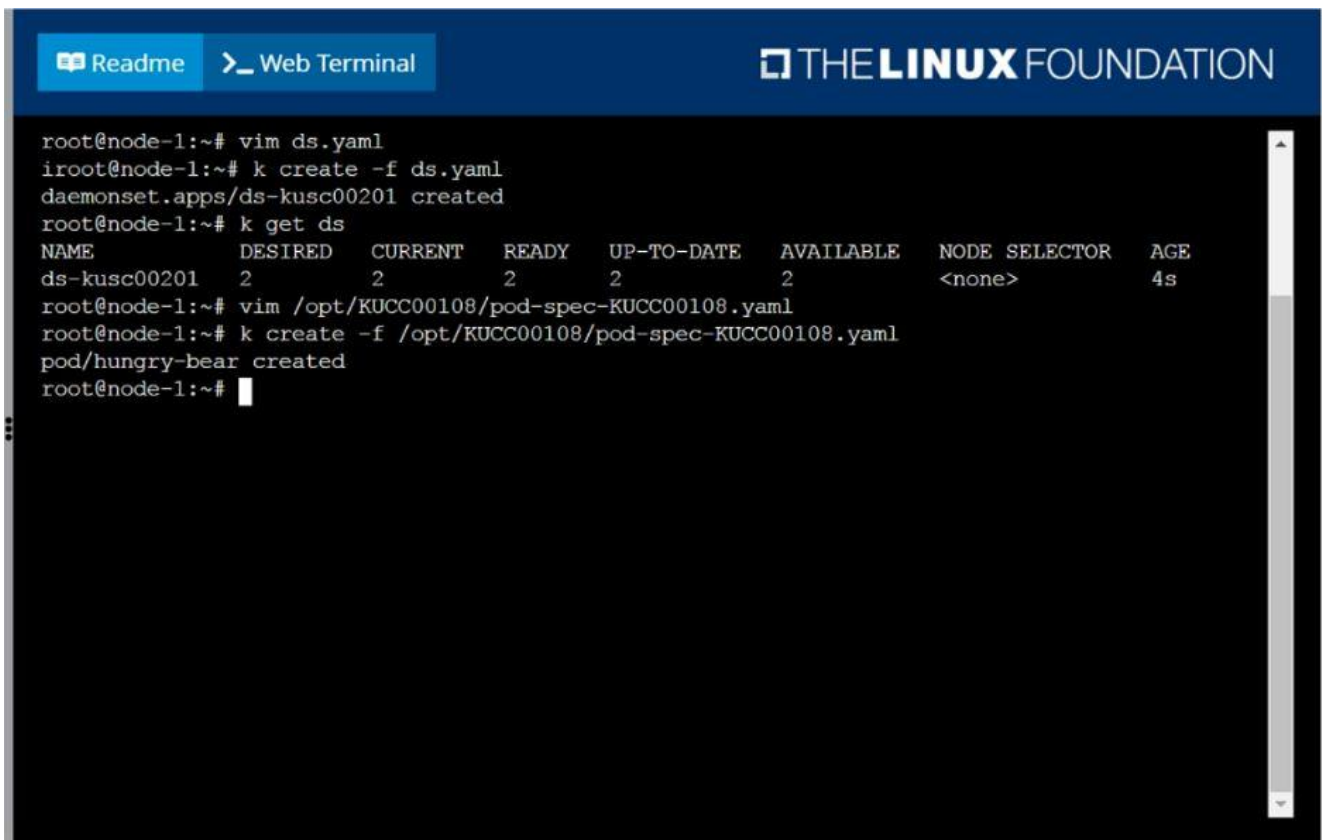


The screenshot shows a web terminal interface with a dark background. At the top, there are two buttons: 'Readme' and 'Web Terminal'. The 'Web Terminal' button is active. The terminal displays a Kubernetes manifest for a pod named 'hungry-bear'. The manifest includes a 'spec' section with 'volumes' and 'containers'. The 'volumes' section defines a 'workdir' volume. The 'containers' section lists two containers: 'checker' and 'create'. The 'checker' container uses the 'alpine' image and runs a command that checks for the existence of '/workdir/calm.txt'. If it exists, it sleeps for 100,000 seconds; otherwise, it exits. The 'create' container also uses the 'alpine' image and runs a command to create the file '/workdir/calm.txt'. Both containers have a 'volumeMounts' section that mounts the 'workdir' volume to '/workdir'.

```

apiVersion: v1
kind: Pod
metadata:
  name: hungry-bear
spec:
  volumes:
    - name: workdir
      emptyDir: {}
  containers:
    - name: checker
      image: alpine
      command: ["/bin/sh", "-c", "if [ -f /workdir/calm.txt ];
        then sleep 100000; else exit 1; fi"]
      volumeMounts:
        - name: workdir
          mountPath: /workdir
    - name: create
      image: alpine
      command: ["/bin/sh", "-c", "touch /workdir/calm.txt"]
      volumeMounts:
        - name: workdir
          mountPath: /workdir
  
```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\4 C.JPG



The screenshot shows a web terminal interface with a dark background. At the top, there are two buttons: 'Readme' and 'Web Terminal'. The 'Web Terminal' button is active. The terminal displays a series of commands and their output. The commands are: 'vim ds.yaml', 'k create -f ds.yaml', 'k get ds', 'vim /opt/KUCC00108/pod-spec-KUCC00108.yaml', and 'k create -f /opt/KUCC00108/pod-spec-KUCC00108.yaml'. The output shows that the pod 'hungry-bear' was created successfully. A table is also displayed, showing the status of the pod.

```

root@node-1:~# vim ds.yaml
iroot@node-1:~# k create -f ds.yaml
daemonset.apps/ds-kusc00201 created
root@node-1:~# k get ds
NAME          DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
ds-kusc00201   2         2         2       2           2           <none>          4s
root@node-1:~# vim /opt/KUCC00108/pod-spec-KUCC00108.yaml
root@node-1:~# k create -f /opt/KUCC00108/pod-spec-KUCC00108.yaml
pod/hungry-bear created
root@node-1:~#
  
```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\4 D.JPG



37. SIMULATION

Score:7%



Context

An existing Pod needs to be integrated into the Kubernetes built-in logging architecture (e.g. kubectl logs).

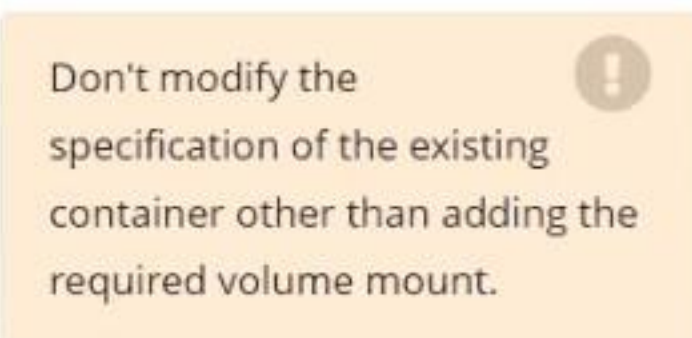
Adding a streaming sidecar container is a good and common way to accomplish this requirement.

Task

Add a sidecar container named sidecar, using the busybox Image, to the existing Pod big- corp-app. The new sidecar container has to run the following command:

```
/bin/sh -c tail -n+1 -f /var/log/big-corp-app.log
```

Use a Volume, mounted at /var/log, to make the log file big-corp-app.log available to the sidecar container.



Answer:

See the solution below.

Explanation:

Solution:

#

```
kubectl get pod big-corp-app -o yaml

#
apiVersion: v1
kind: Pod
metadata:
  name: big-corp-app
spec:
  containers:
  - name: big-corp-app
    image: busybox
    args:
    - /bin/sh
    - -c
    - > i=0;
    while true;
    do
    echo "$(date) INFO $i" >> /var/log/big-corp-app.log;
    i=$((i+1));
    sleep 1;
    done
    volumeMounts:
    - name: logs
      mountPath: /var/log
    - name: count-log-1
      image: busybox
      args: [/bin/sh, -c, 'tail -n+1 -f /var/log/big-corp-app.log']
    volumeMounts:
    - name: logs
      mountPath: /var/log
  volumes:
```

- name: logs

emptyDir: {

}

#

kubectl logs big-corp-app -c count-log-1

38. SIMULATION

You must connect to the correct host. Failure to do so may result in a zero score.

[candidate@base] \$ ssh Cka000058 Context

You manage a WordPress application. Some Pods are not starting because resource requests are too high.

Task

A WordPress application in the relative-fawn namespace consists of:

- . A WordPress Deployment with 3 replicas.

Adjust all Pod resource requests as follows:

- . Divide node resources evenly across all 3 Pods.
- . Give each Pod a fair share of CPU and memory.

Answer:

See the solution below.

Explanation:

MISSING

39. SIMULATION

Score: 5%



Set configuration context:



```
[student@node-1] $ | kube  
ctl config use-context k  
8s
```

Task

From the pod label name=cpu-utilizer, find pods running high CPU workloads and write the name of the pod consuming most CPU to the file /opt/KUTR00401/KUTR00401.txt (which already exists).

Answer:

See the solution below.

Explanation:

Solution:

```
kubectl top -l name=cpu-user -A
```

```
echo 'pod name' >> /opt/KUT00401/KUT00401.txt
```

40. SIMULATION

Create and configure the service front-end-service so it's accessible through NodePort and routes to the existing pod named front-end.

Answer:

See the solution below.

Explanation:

solution

Readme
Web Terminal

```

root@node-1:~# k expose po
error: resource(s) were provided, but no name, label selector, or --all flag specified
See 'kubectl expose -h' for help and examples
root@node-1:~# k expose po  fron-end --name=front-end-service --port=80 --target-port=80 --t
ype=NodePort
Error from server (NotFound): pods "fron-end" not found
root@node-1:~# k expose po  front-end --name=front-end-service --port=80 --target-port=80 --
type=NodePort
service/front-end-service exposed
root@node-1:~# k get svc

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
front-end-service	NodePort	10.103.221.227	<none>	80:31828/TCP	3s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	77d

```

root@node-1:~#

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\8 B.JPG

41. SIMULATION

Create a deployment spec file that will:

- ☞ Launch 7 replicas of the nginx Image with the labelapp_runtime_stage=dev
- ☞ deployment name: kual00201

Save a copy of this spec file to /opt/KUAL00201/spec_deployment.yaml (or /opt/KUAL00201/spec_deployment.json).

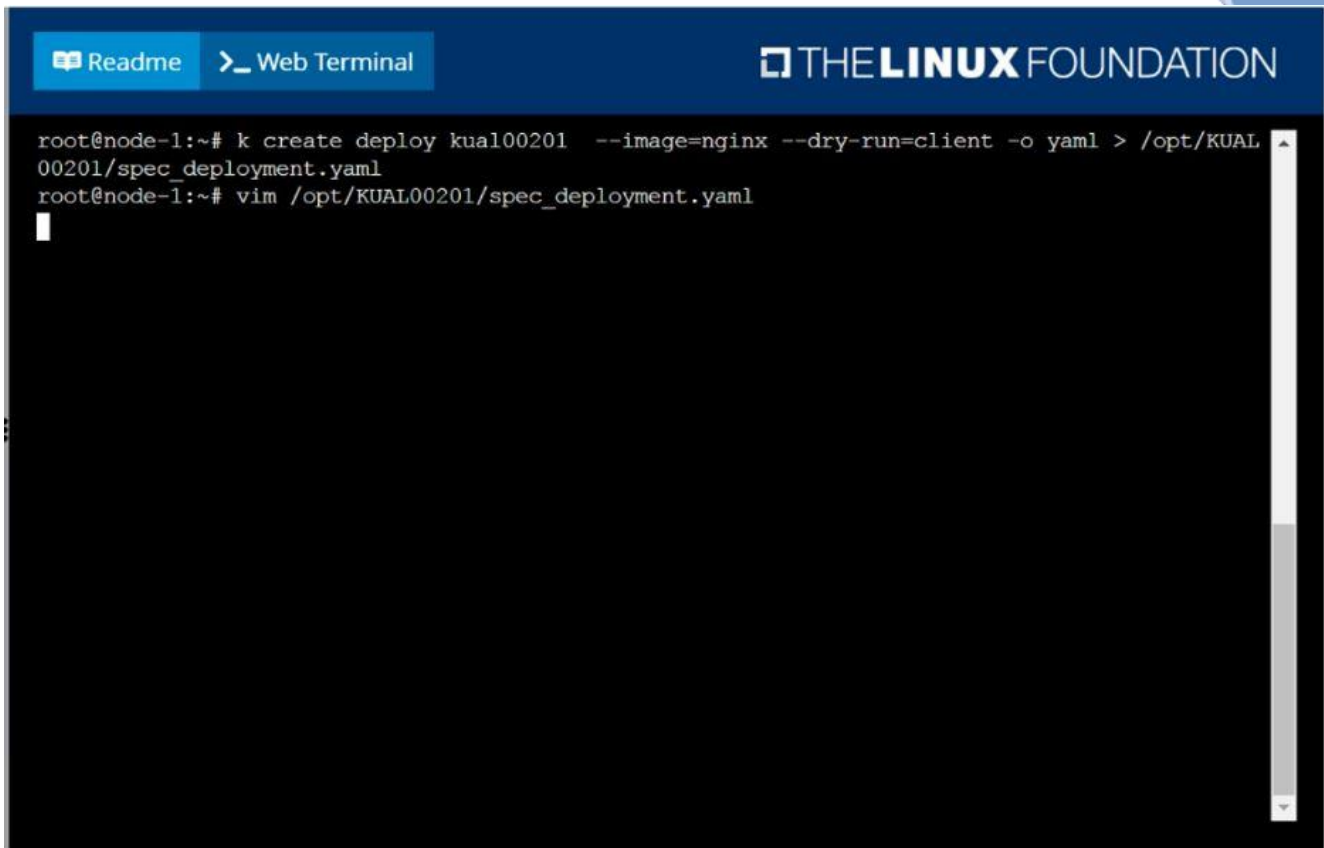
When you are done, clean up (delete) any new Kubernetes API object that you produced during this task.

Answer:

See the solution below.

Explanation:

solution



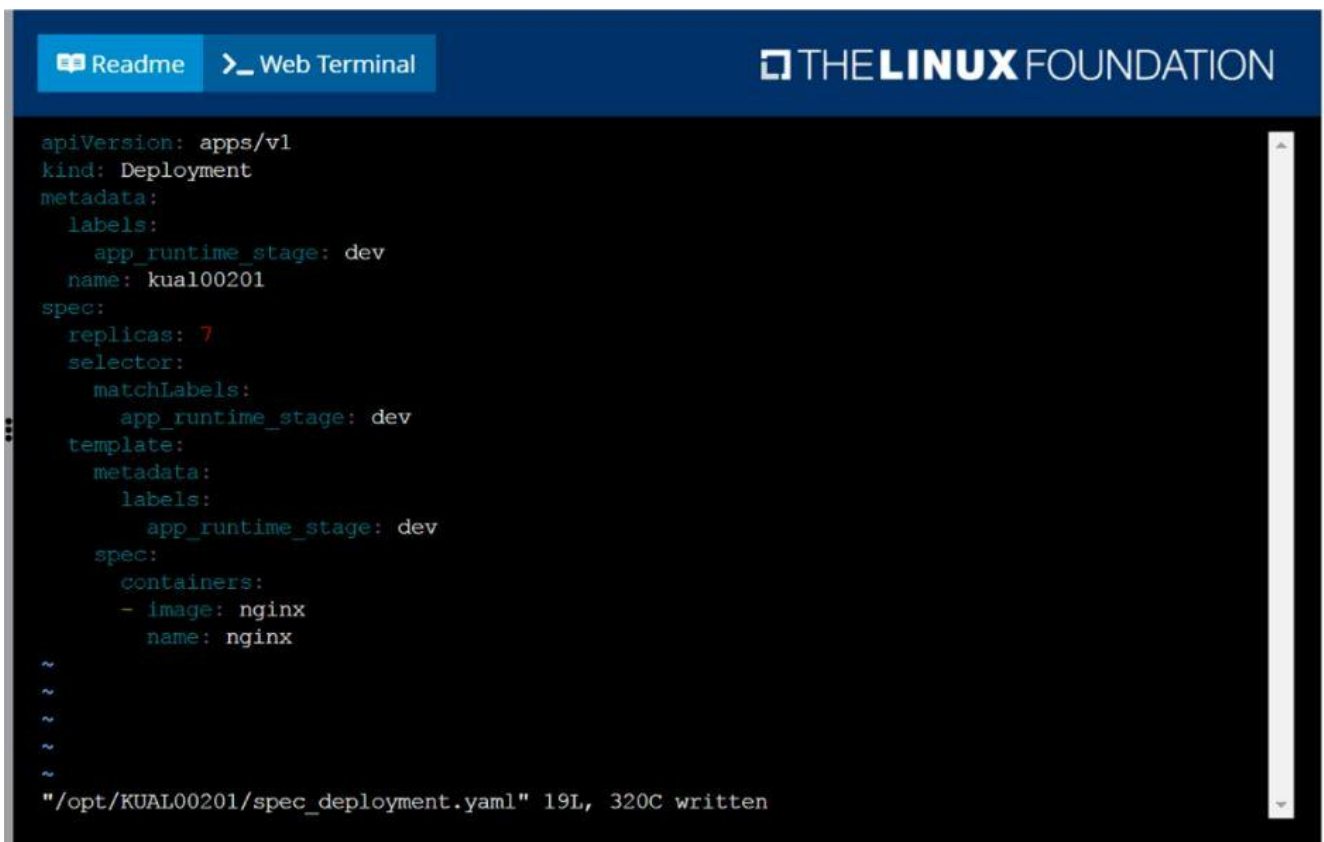
The screenshot shows a web terminal window with a dark background. At the top, there is a blue header bar with a 'Readme' button and a 'Web Terminal' button. To the right of the header is the 'THE LINUX FOUNDATION' logo. The terminal content shows a user at a root prompt on a node-1 machine. The user enters the command 'k create deploy kual00201 --image=nginx --dry-run=client -o yaml > /opt/KUAL00201/spec_deployment.yaml'. The prompt changes to show the file has been created. The user then enters 'vim /opt/KUAL00201/spec_deployment.yaml' and the vim editor opens, showing a blank file with a cursor at the first line.

```

root@node-1:~# k create deploy kual00201 --image=nginx --dry-run=client -o yaml > /opt/KUAL
00201/spec_deployment.yaml
root@node-1:~# vim /opt/KUAL00201/spec_deployment.yaml

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\10 B.JPG



The screenshot shows the same web terminal window as before, but now the vim editor is displaying the contents of the 'spec_deployment.yaml' file. The file contains a Kubernetes Deployment manifest. The 'replicas' field is highlighted in red. At the bottom of the terminal, a message indicates that the file has been written with 19 lines and 320 characters.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app_runtime_stage: dev
  name: kual00201
spec:
  replicas: 7
  selector:
    matchLabels:
      app_runtime_stage: dev
  template:
    metadata:
      labels:
        app_runtime_stage: dev
    spec:
      containers:
      - image: nginx
        name: nginx
~
~
~
~
~
"/opt/KUAL00201/spec_deployment.yaml" 19L, 320C written

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\10 C.JPG



42. SIMULATION

List "nginx-dev" and "nginx-prod" pod and delete those pods

Answer:

See the solution below.

Explanation:

```
kubect1 get pods -o wide
```

```
kubectl delete po "nginx-dev"
```

```
kubectl delete po "nginx-prod"
```

43. SIMULATION

Create a busybox pod and add "sleep 3600" command

Answer:

See the solution below.

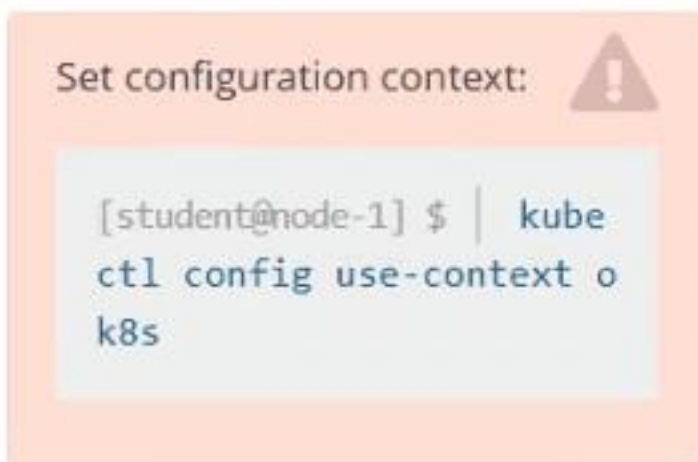
Explanation:

```
kubectl run busybox --image=busybox --restart=Never -- /bin/sh -c
```

```
"sleep 3600"
```

44. SIMULATION

Score:7%



Task

Create a new PersistentVolumeClaim



- Name: pv-volume
- Class: csi-hostpath-sc
- Capacity: 10Mi

Create a new Pod which mounts the PersistentVolumeClaim as a volume:

- Name: web-server
- Image: nginx
- Mount path: /usr/share/nginx/html

Configure the new Pod to have ReadWriteOnce access on the volume.

Finally, using kubectl edit or kubectl patch expand the PersistentVolumeClaim to a capacity of 70Mi and record that change.

Answer:

See the solution below.

Explanation:

Solution:

vi pvc.yaml

storageclass pvc

apiVersion: v1

kind: PersistentVolumeClaim

metadata:

name: pv-volume

spec:

accessModes:

- ReadWriteOnce

volumeMode: Filesystem

resources:

requests:

storage: 10Mi

storageClassName: csi-hostpath-sc

vi pod-pvc.yaml

apiVersion: v1

```
kind: Pod
metadata:
name: web-server
spec:
containers:
- name: web-server
image: nginx
volumeMounts:
- mountPath: "/usr/share/nginx/html"
name: my-volume
volumes:
- name: my-volume
persistentVolumeClaim:
claimName: pv-volume
# craete
kubectl create -f pod-pvc.yaml
#edit
kubectl edit pvc pv-volume --record
```

45. SIMULATION

List all persistent volumes sorted by capacity, saving the full kubectl output to /opt/KUCC00102/volume_list. Use kubectl 's own functionality for sorting the output, and do not manipulate it any further.

Answer:

See the solution below.

Explanation:

solution

Readme
Web Terminal

```

77d
pv0007 7Gi RWO Recycle Available slow
77d
pv0006 8Gi RWO Recycle Available slow
77d
pv0003 10Gi RWO Recycle Available slow
77d
pv0002 11Gi RWO Recycle Available slow
77d
pv0010 13Gi RWO Recycle Available slow
77d
pv0011 14Gi RWO Recycle Available slow
77d
pv0001 16Gi RWO Recycle Available slow
77d
pv0009 17Gi RWO Recycle Available slow
77d
pv0005 18Gi RWO Recycle Available slow
77d
pv0008 19Gi RWO Recycle Available slow
77d
pv0000 21Gi RWO Recycle Available slow
77d
root@node-1:~# k get pv --sort-by=.spec.capacity.storage > /opt/KUCC00102/volume_list
root@node-1:~#

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\2 C.JPG

46. SIMULATION

Score: 7%



Task

Reconfigure the existing deployment front-end and add a port specification named http exposing port 80/tcp of the existing container nginx.

Create a new service named front-end-svc exposing the container port http.



Configure the new service to also expose the individual Pods via a NodePort on the nodes on which they are scheduled.

Answer:

See the solution below.

Explanation:

Solution:

```
kubectl get deploy front-end
```

```
kubectl edit deploy front-end -o yaml
```

```
#port specification named http
```

```
#service.yaml
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
name: front-end-svc
```

```
labels:
```

```
app: nginx
```

```
spec:
```

```
ports:
```

```
- port: 80
```

```
protocol: tcp
```

```
name: http
```

```
selector:
```

```
app: nginx
```

```
type: NodePort
```

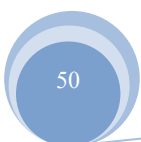
```
# kubectl create -f service.yaml
```

```
# kubectl get svc
```

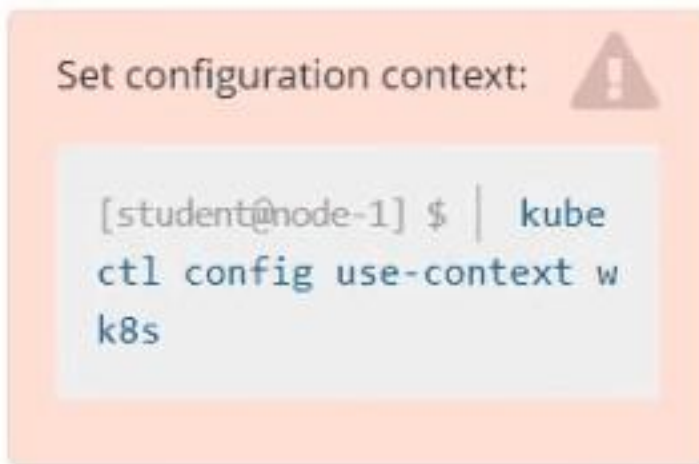
```
# port specification named http
```

```
kubectl expose deployment front-end --name=front-end-svc --port=80 --target-port=80 -- type=NodePort
```

47. SIMULATION



Score: 13%



Task

A Kubernetes worker node, named wk8s-node-0 is in state NotReady. Investigate why this is the case, and perform any appropriate steps to bring the node to a Ready state, ensuring that any changes are made permanent.



Answer:

See the solution below.



Explanation:

Solution:

sudo -i

systemctl status kubelet

systemctl start kubelet

systemctl enable kubelet

48. SIMULATION

Create a pod named kucc8 with a single app container for each of the following images running inside (there may be between 1 and 4 images specified):

nginx + redis + memcached.

Answer:

See the solution below.

Explanation:

solution

The screenshot shows a terminal window with the following content:

```

root@node-1:~# vim ds.yaml
iroot@node-1:~# k create -f ds.yaml
daemonset.apps/ds-kusc00201 created
root@node-1:~# k get ds
NAME                DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
ds-kusc00201        2         2         2       2            2           <none>          4s
root@node-1:~# vim /opt/KUCC00108/pod-spec-KUCC00108.yaml
root@node-1:~# k create -f /opt/KUCC00108/pod-spec-KUCC00108.yaml
pod/hungry-bear created
root@node-1:~# k get po
NAME                READY     STATUS    RESTARTS   AGE
cpu-utilizer-98b9se 1/1       Running   0          5h50m
cpu-utilizer-ab2d3s 1/1       Running   0          5h50m
cpu-utilizer-kipb9a 1/1       Running   0          5h50m
ds-kusc00201-2r2k9  1/1       Running   0          4m50s
ds-kusc00201-hzm9q  1/1       Running   0          4m50s
foo                 1/1       Running   0          5h52m
front-end           1/1       Running   0          5h52m
hungry-bear         1/1       Running   0          42s
webserver-84c55967f4-qzjcv 1/1       Running   0          6h7m
webserver-84c55967f4-t479l 1/1       Running   0          6h7m
root@node-1:~# k run nginx --image=nginx --dry-run=client -o yaml > nginx.yaml
root@node-1:~# vim nginx.yaml

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\5 B.JPG



49. SIMULATION

Score: 7%



Task

Given an existing Kubernetes cluster running version 1.20.0, upgrade all of the Kubernetes control plane and node components on the master node only to version 1.20.1.

Be sure to drain the master node before upgrading it and uncordon it after the upgrade.



You can ssh to the master node using: 

```
[student@node-1] $ | ssh
mk8s-master-0
```

You can assume elevated privileges on the master node with the following command:

```
[student@mk8s-master-0] |
$
sudo -i
```

You are also expected to upgrade kubelet and kubectl on the master node.

Do not upgrade the worker nodes, etcd, the container manager, the CNI plugin, the DNS service or any other addons. 

Answer:

See the solution below.

Explanation:

SOLUTION:

```
[student@node-1] > ssh ek8s
```

```
kubectl cordon k8s-master
```

```
kubectldrain k8s-master --delete-local-data --ignore-daemonsets --force
apt-get install kubeadm=1.20.1-00 kubelet=1.20.1-00 kubectld=1.20.1-00 --
disableexcludes=kubernetes
kubeadm upgrade apply 1.20.1 --etcd-upgrade=false
systemctl daemon-reload
systemctl restart kubelet kubectld
uncordon k8s-master
```

50. SIMULATION

Create a pod with environment variables as var1=value1. Check the environment variable in pod

Answer:

See the solution below.

Explanation:

```
kubectldrun nginx --image=nginx --restart=Never --env=var1=value1
```

then

```
kubectldexec -it nginx -- env
```

or

```
kubectldexec -it nginx -- sh -c 'echo $var1'
```

or

```
kubectlddescribe po nginx | grep value1
```

51. SIMULATION



Quick Reference

Documentation **Ingress** Service, Deployment

You must connect to the correct host .

Failure to do so may result in a zero score.

```
[candidate@base] $ ssh cka000024
```

Task

Create a new Ingress resource as follows:

- . Name: echo
- . Namespace : sound-repeater
- . Exposing Service echoserver-service on http://example.org/echo using Service port 8080

The availability of Service echoserver-service can be checked

i

using the following command, which should return 200 :

```
[candidate@cka000024] $ curl -o /dev/null -s -w "%{http_code}\n" http://example.org/echo
```

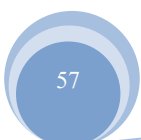
Answer:

See the solution below.

Explanation:

Task Summary

Create an Ingress named echo in the sound-repeater namespace that:





Routes requests to /echo on host example.org

Forwards traffic to service echoserver-service

Uses service port 8080

Verification should return HTTP 200 using curl

Step-by-Step Answer

1# SSH into the correct node

As shown in the image:

bash

CopyEdit

ssh cka000024

Skipping this will result in a ZERO score!

2# Verify the namespace and service

Ensure the sound-repeater namespace and echoserver-service exist:

kubectl get svc -n sound-repeater

Look for:

echoserver-service ClusterIP ... 8080/TCP

3# Create the Ingress manifest

Create a YAML file: echo-ingress.yaml

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

name: echo

namespace: sound-repeater

annotations:

nginx.ingress.kubernetes.io/rewrite-target: /\$1

spec:

rules:

- host: example.org

https:

paths:



- path: /echo

pathType: Prefix

backend:

service:

name: echoserver-service

port:

number: 8080

4# Apply the Ingress resource

kubectl apply -f echo-ingress.yaml

5# Test with curl as instructed

Use the exact verification command:

curl -o /dev/null -s -w "%{http_code}\n" https://example.org/echo

You should see:

200

Final Answer Summary

ssh cka000024

kubectl get svc -n sound-repeater

Create the Ingress YAML

cat < echo-ingress.yaml

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

name: echo

namespace: sound-repeater

annotations:

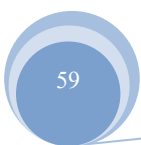
nginx.ingress.kubernetes.io/rewrite-target: /\$1

spec:

rules:

- host: example.org

https:



paths:

- path: /echo

pathType: Prefix

backend:

service:

name: echoserver-service

port:

number: 8080

EOF

kubectl apply -f echo-ingress.yaml

curl -o /dev/null -s -w "%{http_code}\n" https://example.org/echo

52. SIMULATION

Create a snapshot of the etcd instance running at `https://127.0.0.1:2379`, saving the snapshot to the file path `/srv/data/etcd-snapshot.db`.

The following TLS certificates/key are supplied for connecting to the server with `etcdctl`:

☞ CA certificate: `/opt/KUCM00302/ca.crt`

☞ Client certificate: `/opt/KUCM00302/etcd-client.crt`

☞ Client key: `/opt/KUCM00302/etcd-client.key`

Answer:

See the solution below.

Explanation:

solution

Readme
Web Terminal
THE LINUX FOUNDATION

```

root@node-1:~# ETCDCCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 --cacert=/opt/KUCM00302/ca.crt --cert=/opt/KUCM00302/etcd-client.crt --key=/opt/KUCM00302/etcd-client.key snapshot save /srv/data/etcd-snapshot.db
{"level":"info","ts":1598530470.8313155,"caller":"snapshot/v3_snapshot.go:110","msg":"created temporary db file","path":"/srv/data/etcd-snapshot.db.part"}
{"level":"warn","ts":"2020-08-27T12:14:30.838Z","caller":"clientv3/retry_interceptor.go:116","msg":"retry stream intercept"}
{"level":"info","ts":1598530470.8388612,"caller":"snapshot/v3_snapshot.go:121","msg":"fetching snapshot","endpoint":"https://127.0.0.1:2379"}
{"level":"info","ts":1598530470.8570414,"caller":"snapshot/v3_snapshot.go:134","msg":"fetched snapshot","endpoint":"https://127.0.0.1:2379","took":0.025676157}
{"level":"info","ts":1598530470.8571067,"caller":"snapshot/v3_snapshot.go:143","msg":"saved","path":"/srv/data/etcd-snapshot.db"}
Snapshot saved at /srv/data/etcd-snapshot.db
root@node-1:~#

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\18 C.JPG

53. SIMULATION

Score: 4%



Task

Set the node named ek8s-node-1 as unavailable and reschedule all the pods running on it.

Answer:

See the solution below.



Explanation:

SOLUTION:

```
[student@node-1] > ssh ek8s
```

```
kubectl cordon ek8s-node-1
```

```
kubectl drain ek8s-node-1 --delete-local-data --ignore-daemonsets --force
```

54. SIMULATION

Scale the deployment webserver to 6 pods.

Answer:

See the solution below.

Explanation:

solution

```

Readme  Web Terminal  THE LINUX FOUNDATION

root@node-1:~# k scale deploy webserver --replicas=6
deployment.apps/webserver scaled
root@node-1:~# k get deploy
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
nginx-app     3/3     3            3           29m
webserver     6/6     6            6           6h50m
root@node-1:~#
    
```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\14 B.JPG

55. SIMULATION

A Kubernetes worker node, named wk8s-node-0 is in state NotReady. Investigate why this is the case, and

perform any appropriate steps to bring the node to a Ready state, ensuring that any changes are made permanent.

You can ssh to the failed node using: [student@node-1] \$ | ssh Wk8s-node-0

You can assume elevated privileges on the node with the following command: [student@w8ks-node-0] \$ |

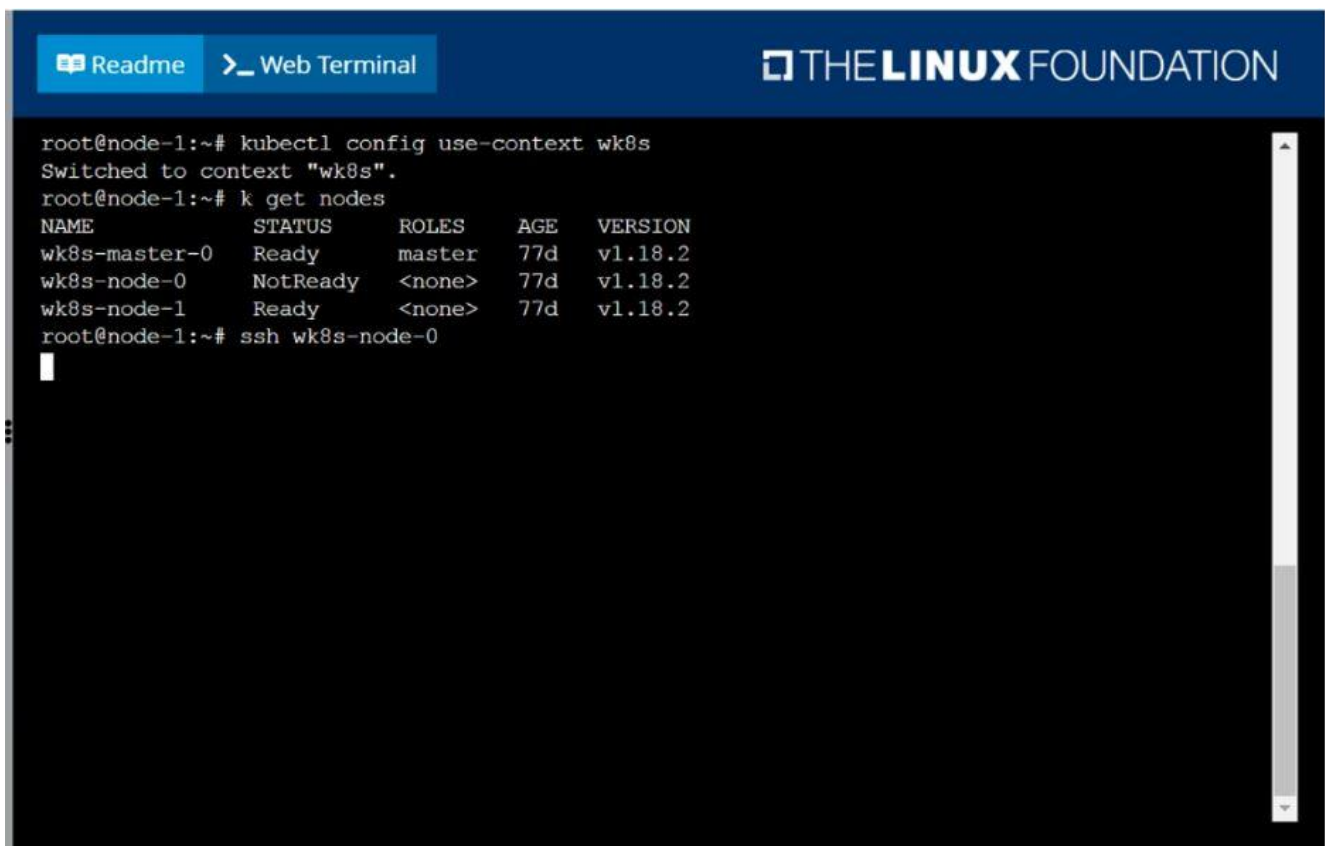
sudo -i

Answer:

See the solution below.

Explanation:

solution



```

root@node-1:~# kubectl config use-context wk8s
Switched to context "wk8s".
root@node-1:~# k get nodes
NAME             STATUS    ROLES    AGE   VERSION
wk8s-master-0    Ready     master   77d   v1.18.2
wk8s-node-0      NotReady  <none>    77d   v1.18.2
wk8s-node-1      Ready     <none>    77d   v1.18.2
root@node-1:~# ssh wk8s-node-0

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\20 C.JPG

Readme
Web Terminal
THE **LINUX** FOUNDATION

```

wk8s-node-0    NotReady    <none>    77d    v1.18.2
wk8s-node-1    Ready      <none>    77d    v1.18.2
root@node-1:~# ssh wk8s-node-0
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.4.0-1109-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Are you ready for Kubernetes 1.19? It's nearly here! Try RC3 with
   sudo snap install microk8s --channel=1.19/candidate --classic

   https://microk8s.io/ has docs and details.

4 packages can be updated.
1 update is a security update.

New release '18.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

student@wk8s-node-0:~$ sudo -i
root@wk8s-node-0:~# systemctl restart kubelet
root@wk8s-node-0:~# systemctl enable kubelet

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\20 D.JPG

Readme
Web Terminal
THE **LINUX** FOUNDATION

```

https://microk8s.io/ has docs and details.

4 packages can be updated.
1 update is a security update.

New release '18.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

student@wk8s-node-0:~$ sudo -i
root@wk8s-node-0:~# systemctl restart kubelet
root@wk8s-node-0:~# systemctl enable kubelet
Created symlink from /etc/systemd/system/multi-user.target.wants/kubelet.service to /lib/sys
temd/systemd/kubelet.service.
root@wk8s-node-0:~# exit
logout
student@wk8s-node-0:~$ exit
logout
Connection to 10.250.5.34 closed.
root@node-1:~# k get nodes
NAME          STATUS    ROLES    AGE    VERSION
wk8s-master-0 Ready    master   77d    v1.18.2
wk8s-node-0   Ready    <none>   77d    v1.18.2
wk8s-node-1   Ready    <none>   77d    v1.18.2
root@node-1:~#

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\20 E.JPG



56. SIMULATION

You must connect to the correct host. Failure to do so may result in a zero score.

[candidate@base] \$ ssh Cka000051 Context

You manage a WordPress application. Some Pods are not starting because resource requests are too high.

Your task is to prepare a Linux system for Kubernetes . Docker is already installed, but you need to configure it for kubeadm .

Task

Complete these tasks to prepare the system for Kubernetes : Set up cri-dockerd :

. Install the Debian package

~/cri-dockerd_0.3.9.3-0.ubuntu-jammy_amd64.deb

Debian packages are installed using dpkg .

. Enable and start the cri-docker service Configure these system parameters:

. Set net.bridge.bridge-nf-call-iptables to 1

Answer:

See the solution below.

Explanation:

MISSING

57. SIMULATION

You must connect to the correct host. Failure to do so may result in a zero score.

[candidate@base] \$ ssh Cka000047 Task

A MariaDB Deployment in the mariadb namespace has been deleted by mistake. Your task is to restore the Deployment ensuring data persistence. Follow these steps:

Create a PersistentVolumeClaim (PVC) named mariadb in the mariadb namespace with the following specifications: Access mode ReadWriteOnce

Storage 250Mi

You must use the existing retained PersistentVolume (PV). Failure to do so will result in a reduced score.

There is only one existing PersistentVolume .

Edit the MariaDB Deployment file located at ~/mariadb-deployment.yaml to use PVC you created in the



previous step.

Apply the updated Deployment file to the cluster.

Ensure the MariaDB Deployment is running and stable.

Answer:

See the solution below.

Explanation:

MISSING

58. SIMULATION

You must connect to the correct host. Failure to do so may result in a zero score.

[candidate@base] \$ ssh Cka000060 Task

Install Argo CD in the cluster by performing the following tasks: Add the official Argo CD Helm repository with the name argo

The Argo CD CRDs have already been pre-installed in the cluster

Generate a template of the Argo CD Helm chart version 7.7.3 for the argocd namespace and save it to ~/argo-helm.yaml . Configure the chart to not install CRDs.

Answer:

See the solution below.

Task Summary

- * SSH into cka000060
- * Add the Argo CD Helm repo named argo
- * Generate a manifest (~/.argo-helm.yaml) for Argo CD version 7.7.3
- * Target namespace: argocd
- * Do not install CRDs
- * Just generate, don't install

Step-by-Step Solution

1## SSH into the correct host

ssh cka000060

Required - skipping this = zero score

2## Add the Argo CD Helm repository





```
helm repo add argo https://argoproj.github.io/argo-helm
```

```
helm repo update
```

```
# This adds the official Argo Helm chart source.
```

```
3## Generate Argo CD Helm chart template (version 7.7.3)
```

Use the helm template command to generate a manifest and write it to ~/argo-helm.yaml.

```
helm template argocd argo/argo-cd \
```

```
--version 7.7.3 \
```

```
--namespace argocd \
```

```
--set crds.install=false \
```

```
&gt; ~/argo-helm.yaml
```

```
* argocd # Release name (can be anything; here it's same as the namespace)
```

```
* --set crds.install=false # Disables CRD installation
```

```
* &gt; ~/argo-helm.yaml # Save to required file
```

```
# 4## Verify the generated file (optional but smart)
```

```
head ~/argo-helm.yaml
```

Check that it contains valid Kubernetes YAML and does not include CRDs.

```
# Final Command Summary
```

```
ssh cka000060
```

```
helm repo add argo https://argoproj.github.io/argo-helm
```

```
helm repo update
```

```
helm template argocd argo/argo-cd \
```

```
--version 7.7.3 \
```

```
--namespace argocd \
```

```
--set crds.install=false \
```

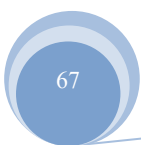
```
&gt; ~/argo-helm.yaml
```

```
head ~/argo-helm.yaml # Optional verification
```

59. SIMULATION

Create a file:

/opt/KUCC00302/kucc00302.txt that lists all pods that implement service baz in namespace development.





The format of the file should be one pod name per line.

Answer:

See the solution below.

Explanation:

solution

```

root@node-1:~#
root@node-1:~# k describe svc baz -n development
Name:          baz
Namespace:     development
Labels:        <none>
Annotations:   <none>
Selector:      name=foo
Type:          ClusterIP
IP:            10.104.252.175
Port:          <unset> 80/TCP
TargetPort:    9376/TCP
Endpoints:     10.244.1.5:9376,10.244.2.3:9376,10.244.2.6:9376
Session Affinity: None
Events:        <none>
root@node-1:~# k get po -l name=foo -n development
NAME                                READY   STATUS    RESTARTS   AGE
pod-kucc00302-847878                1/1     Running   0           6h35m
pod-kucc00302-983457                1/1     Running   0           6h35m
pod-kucc00302-985953                1/1     Running   0           6h35m
root@node-1:~# k get po -l name=foo -n development -o NAME
pod/pod-kucc00302-847878
pod/pod-kucc00302-983457
pod/pod-kucc00302-985953
root@node-1:~# k get po -l name=foo -n development -o NAME > /opt/KUCC00302/kucc00302.txt
root@node-1:~# vim /opt/KUCC00302/kucc00302.txt
    
```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\11 B.JPG





60. SIMULATION

List all the pods sorted by name

Answer:

See the solution below.

Explanation:

```
kubectl get pods --sort-by=.metadata.name
```

61. SIMULATION

List all the pods sorted by created timestamp

Answer:

See the solution below.

Explanation:

```
kubect1 get pods--sort-by=.metadata.creationTimestamp
```

62. SIMULATION

Quick Reference

ConfigMaps,

Documentation Deployments, Namespace

You must connect to the correct host . Failure to do so may result in a zero score.

[candidate@base] \$ ssh cka000048b Task

An NGINX Deployment named nginx-static is running in the nginx-static namespace. It is configured using a ConfigMap named nginx-config .

First, update the nginx-config ConfigMap to also allow TLSv1.2. connections. You may re-create, restart, or scale resources as necessary.

You can use the following command to test the changes: [candidate@cka000048b] \$ curl -- tls-max 1.2 https://web.k8s.local

Answer:

See the solution below.

Explanation:



Task Summary

SSH into cka000048b

Update the nginx-config ConfigMap in the nginx-static namespace to allow TLSv1.2

Ensure the nginx-static Deployment picks up the new config

Verify the change using the provided curl command

Step-by-Step Instructions

Step 1: SSH into the correct host

```
ssh cka000048b
```

Step 2: Get the ConfigMap

```
kubectl get configmap nginx-config -n nginx-static -o yaml > nginx-config.yaml
```

Open the file for editing:

```
nano nginx-config.yaml
```

Look for the TLS configuration in the data field. You are likely to find something like:

```
ssl_protocols TLSv1.3;
```

Modify it to include TLSv1.2 as well:

```
ssl_protocols TLSv1.2 TLSv1.3;
```

Save and exit the file.

Now update the ConfigMap:

```
kubectl apply -f nginx-config.yaml
```

Step 3: Restart the NGINX pods to pick up the new ConfigMap

Pods will not reload a ConfigMap automatically unless it's mounted in a way that supports dynamic reload and the app is watching for it (NGINX typically doesn't by default).

The safest way is to restart the pods:

Option 1: Roll the deployment

```
kubectl rollout restart deployment nginx-static -n nginx-static
```

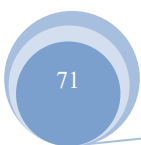
Option 2: Delete pods to force recreation

```
kubectl delete pod -n nginx-static -l app=nginx-static
```

Step 4: Verify using curl

Use the provided curl command to confirm that TLS 1.2 is accepted:

```
curl --tls-max 1.2 https://web.k8s.local
```



A successful response means the TLS configuration is correct.

Final Command Summary

```
ssh cka000048b
```

```
kubectl get configmap nginx-config -n nginx-static -o yaml > nginx-config.yaml
```

```
nano nginx-config.yaml # Modify to include "ssl_protocols TLSv1.2 TLSv1.3;"
```

```
kubectl apply -f nginx-config.yaml
```

```
kubectl rollout restart deployment nginx-static -n nginx-static
```

or

```
kubectl delete pod -n nginx-static -l app=nginx-static
```

```
curl --tls-max 1.2 https://web.k8s.local
```

63. SIMULATION

Monitor the logs of pod foo and:

- ☞ Extract log lines corresponding to error unable-to-access-website
- ☞ Write them to/opt/KULM00201/foo

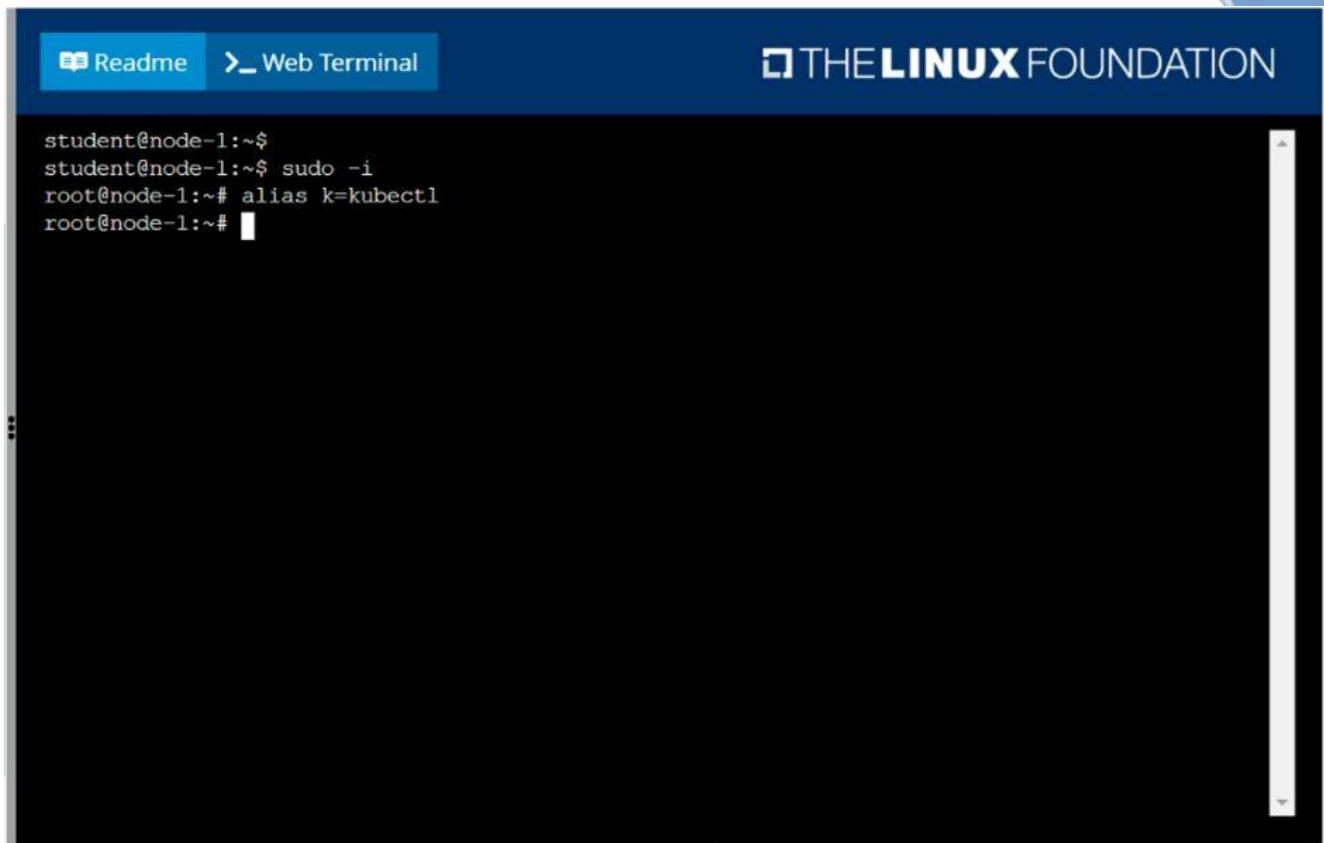


Answer:

See the solution below.

Explanation:

solution

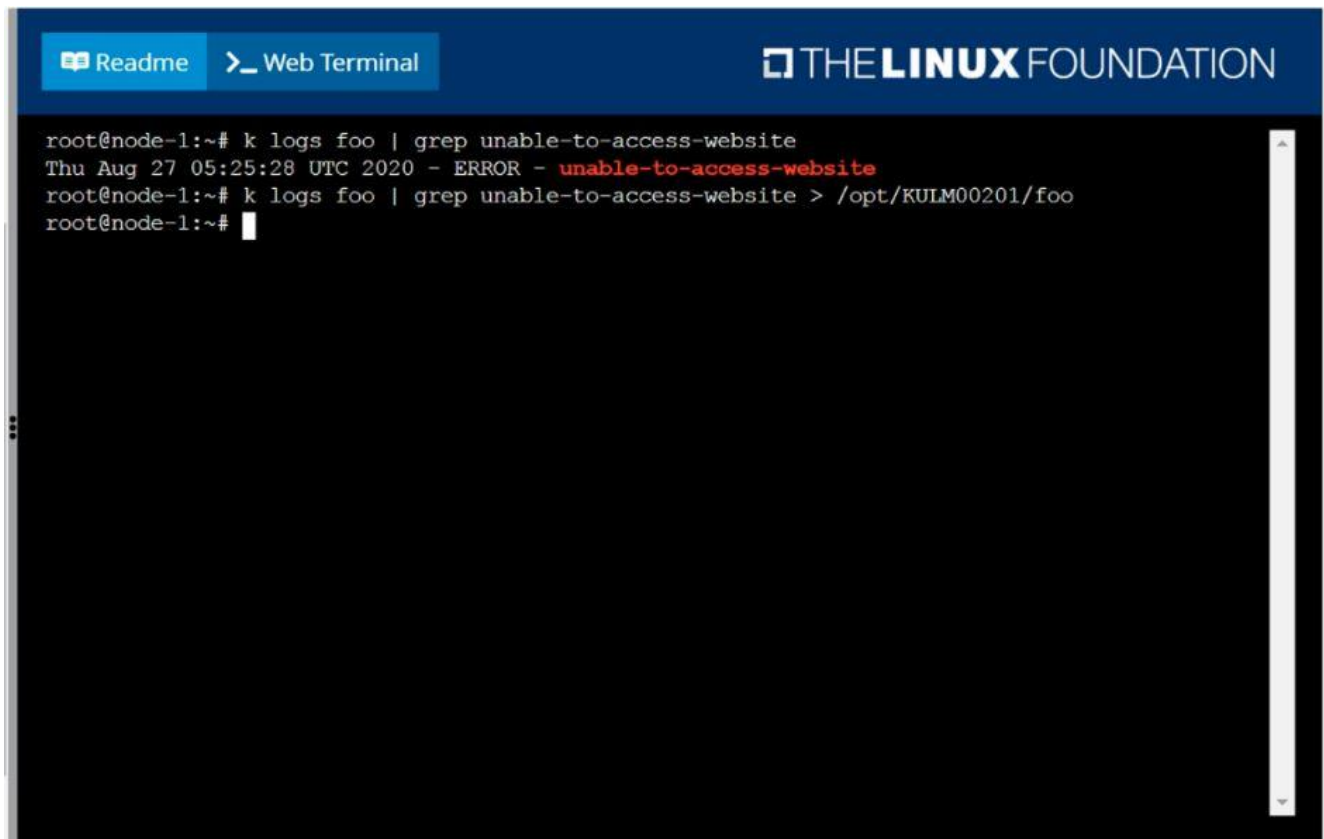


```

student@node-1:~$
student@node-1:~$ sudo -i
root@node-1:~# alias k=kubectl
root@node-1:~#

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\1 B.JPG



```

root@node-1:~# k logs foo | grep unable-to-access-website
Thu Aug 27 05:25:28 UTC 2020 - ERROR - unable-to-access-website
root@node-1:~# k logs foo | grep unable-to-access-website > /opt/KULM00201/foo
root@node-1:~#

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\1 C.JPG



64. SIMULATION

Configure the kubelet systemd- managed service, on the node labelled with name=wk8s- node-1, to launch a pod containing a single container of Image httpd named webtool automatically. Any spec files required should be placed in the /etc/kubernetes/manifests directory on the node.

You can ssh to the appropriate node using: [student@node-1] \$ ssh wk8s-node-1

You can assume elevated privileges on the node with the following command: [student@wk8s-node-1] \$ | sudo -i

Answer:

See the solution below.

Explanation:

solution

```

root@node-1:~#
root@node-1:~# kubectl config use-context wk8s
Switched to context "wk8s".
root@node-1:~# ssh wk8s-node-1
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.4.0-1109-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Are you ready for Kubernetes 1.19? It's nearly here! Try RC3 with
   sudo snap install microk8s --channel=1.19/candidate --classic

   https://microk8s.io/ has docs and details.

4 packages can be updated.
1 update is a security update.

New release '18.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

student@wk8s-node-1:~$ sudo -i
root@wk8s-node-1:~# vim /var/lib/kubelet/config.yaml

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\21 C.JPG

Readme
Web Terminal

```

    clientCAFile: /etc/kubernetes/pki/ca.crt
authorization:
  mode: Webhook
  webhook:
    cacheAuthorizedTTL: 0s
    cacheUnauthorizedTTL: 0s
clusterDNS:
- 10.96.0.10
clusterDomain: cluster.local
cpuManagerReconcilePeriod: 0s
evictionPressureTransitionPeriod: 0s
fileCheckFrequency: 0s
healthzBindAddress: 127.0.0.1
healthzPort: 10248
httpCheckFrequency: 0s
imageMinimumGCAge: 0s
kind: KubeletConfiguration
nodeStatusReportFrequency: 0s
nodeStatusUpdateFrequency: 0s
rotateCertificates: true
runtimeRequestTimeout: 0s
staticPodPath: /etc/kubernetes/manifests
streamingConnectionIdleTimeout: 0s
syncFrequency: 0s
:wc

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\21 D.JPG

Readme
Web Terminal

```

root@node-1:~# ssh wk8s-node-1
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.4.0-1109-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Are you ready for Kubernetes 1.19? It's nearly here! Try RC3 with
   sudo snap install microk8s --channel=1.19/candidate --classic

   https://microk8s.io/ has docs and details.

4 packages can be updated.
1 update is a security update.

New release '18.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

student@wk8s-node-1:~$ sudo -i
root@wk8s-node-1:~# vim /var/lib/kubelet/config.yaml
root@wk8s-node-1:~# cd /etc/kubernetes/manifests
root@wk8s-node-1:/etc/kubernetes/manifests#
root@wk8s-node-1:/etc/kubernetes/manifests# vim pod.yaml

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\21 E.JPG





65. SIMULATION

Ensure a single instance of pod nginx is running on each node of the Kubernetes cluster where nginx also represents the Image name which has to be used. Do not override any taints currently in place.

Use DaemonSet to complete this task and use ds-kusc00201 as DaemonSet name.

Answer:

See the solution below.

Explanation:

solution

The screenshot shows a web terminal window with a dark blue header. On the left, there are two tabs: 'Readme' and 'Web Terminal'. On the right, the text 'THE LINUX FOUNDATION' is displayed. The terminal content shows a user at 'root@node-1:~#' running the command 'vim ds.yaml'. The vim editor is open, showing a single line 'i' on the first line, with a cursor at the end of the line. A vertical scrollbar is visible on the right side of the terminal window.

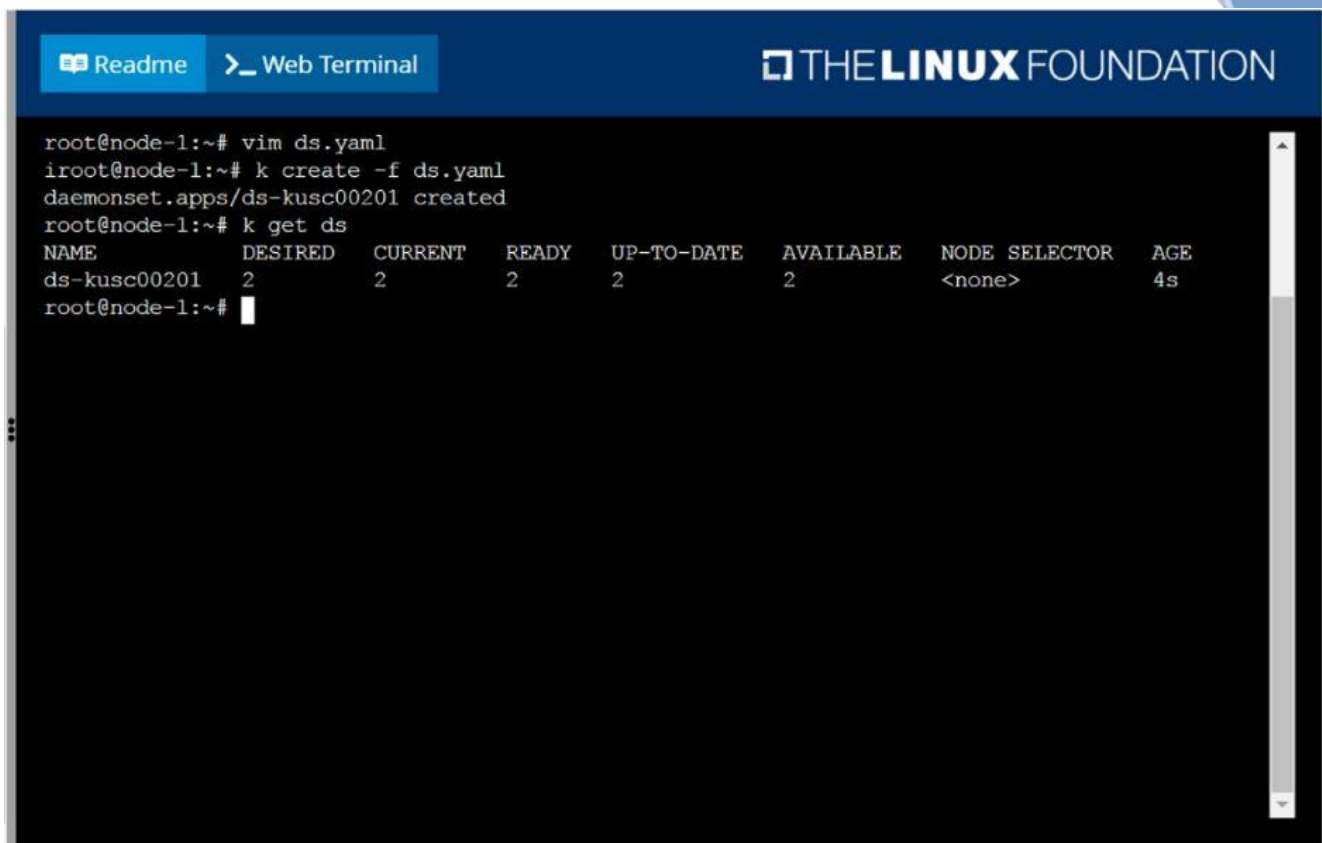
F:\Work\Data Entry Work\Data Entry\20200827\CKA\3 B.JPG



ATION



ATION



```

root@node-1:~# vim ds.yaml
root@node-1:~# k create -f ds.yaml
daemonset.apps/ds-kusc00201 created
root@node-1:~# k get ds

```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE SELECTOR	AGE
ds-kusc00201	2	2	2	2	2	<none>	4s

```

root@node-1:~#

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\3 E.JPG

66. SIMULATION

Score: 7%



Task

Create a new NetworkPolicy named allow-port-from-namespace in the existing namespace echo. Ensure that the new NetworkPolicy allows Pods in namespace my-app to connect to port 9000 of Pods in namespace echo.



Further ensure that the new NetworkPolicy:

- does not allow access to Pods, which don't listen on port 9000
- does not allow access from Pods, which are not in namespace my-app

Answer:

See the solution below.

Explanation:

Solution:

```
#network.yaml

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-port-from-namespace
  namespace: internal
spec:
  podSelector:
    matchLabels: {
    }
  policyTypes:
    - Ingress
  ingress:
    - from:
      - podSelector: {
      }
    ports:
      - protocol: TCP
        port: 8080
#spec.podSelector namespace pod
kubectl create -f network.yaml
```

67. SIMULATION

List the nginx pod with custom columns POD_NAME and POD_STATUS

Answer:

See the solution below.

Explanation:

```
kubectl get po -o=custom-columns="POD_NAME:.metadata.name,  
POD_STATUS:.status.containerStatuses[].state"
```

68. SIMULATION

Schedule a pod as follows:

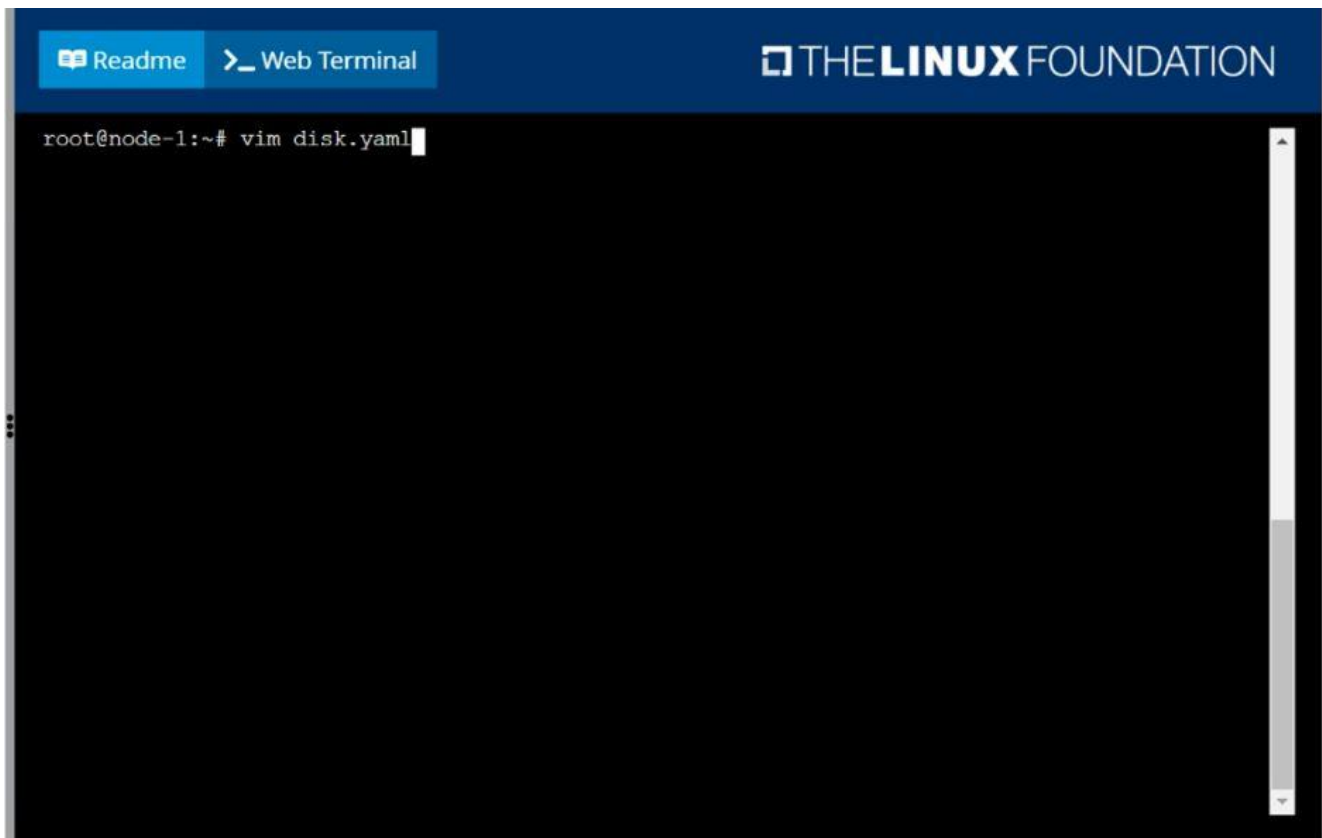
- ☞ Name: nginx-kusc00101
- ☞ Image: nginx
- ☞ Node selector: disk=ssd

Answer:

See the solution below.

Explanation:

solution



[illegible]

Readme

Web Terminal

THE **LINUX** FOUNDATION

```
root@node-1:~# vim disk.yaml
root@node-1:~# k create -f disk.yaml
pod/nginx-kusc00101 created
root@node-1:~# k get po
NAME                                READY   STATUS    RESTARTS   AGE
cpu-utilizer-98b9se                1/1     Running   0           5h59m
cpu-utilizer-ab2d3s                1/1     Running   0           5h59m
cpu-utilizer-kipb9a                1/1     Running   0           5h59m
ds-kusc00201-2r2k9                 1/1     Running   0           13m
ds-kusc00201-hzm9q                 1/1     Running   0           13m
foo                                1/1     Running   0           6h1m
front-end                           1/1     Running   0           6h1m
hungry-bear                         1/1     Running   0           9m37s
kucc8                               3/3     Running   0           7m37s
nginx-kusc00101                    1/1     Running   0           9s
webserver-84c55967f4-qzjcv         1/1     Running   0           6h16m
webserver-84c55967f4-t479l         1/1     Running   0           6h16m
root@node-1:~#
```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\6 D.JPG

69. SIMULATION

Score: 4%



Task

Check to see how many nodes are ready schedulable (not including nodes tainted NoSchedule) and write the number to /opt/KUSC00402/kusc00402.txt.

Answer:

See the solution below.

Explanation:

Solution:

```
kubectl describe nodes | grep ready|wc -l
```

```
kubectl describe nodes | grep -i taint | grep -i noschedule |wc -l
```

```
echo 3 > /opt/KUSC00402/kusc00402.txt
```

```
#
```

```
kubectl get node | grep -i ready |wc -l
```

```
# taintsnoSchedule
```

```
kubectl describe nodes | grep -i taints | grep -i noschedule |wc -l
```

```
#
```

```
echo 2 > /opt/KUSC00402/kusc00402.txt
```

70. SIMULATION



You must connect to the correct host. Failure to do so may result in a zero score.

[candidate@base] \$ ssh Cka000056 Task

Review and apply the appropriate NetworkPolicy from the provided YAML samples.

Ensure that the chosen NetworkPolicy is not overly permissive, but allows communication between the frontend and backend Deployments, which run in the frontend and backend namespaces respectively.

First, analyze the frontend and backend Deployments to determine the specific requirements for the NetworkPolicy that needs to be applied.

Next, examine the NetworkPolicy YAML samples located in the ~/netpol folder.

Failure to comply may result in a reduced score.

Do not delete or modify the provided samples. Only apply one of them.

Finally, apply the NetworkPolicy that enables communication between the frontend and backend Deployments, without being overly permissive.

Answer:

See the solution below.

Explanation:

Task Summary

Connect to host cka000056

Review existing frontend and backend Deployments

Choose one correct NetworkPolicy from the ~/netpol directory

The policy must:

Allow traffic only from the frontend Deployment to the backend Deployment

Avoid being overly permissive

Apply the correct NetworkPolicy without modifying any sample files

Step-by-Step Instructions

Step 1: SSH into the correct node

ssh cka000056

Step 2: Inspect the frontend Deployment

Check the labels used in the frontend Deployment:

kubectl get deployment -n frontend -o yaml

Look under metadata.labels or spec.template.metadata.labels. Note the app or similar label (e.g., app:



frontend).

Step 3: Inspect the backend Deployment

```
kubectl get deployment -n backend -o yaml
```

Again, find the labels assigned to the pods (e.g., app: backend).

Step 4: List and review the provided NetworkPolicies

List the available files:

```
ls ~/netpol
```

Check the contents of each policy file:

```
cat ~/netpol/.yaml
```

Look for a policy that:

Has kind: NetworkPolicy

Applies to the backend namespace

Uses a podSelector that matches the backend pods

Includes an ingress.from rule that references the frontend namespace using a namespaceSelector (and optionally a podSelector)

Does not allow traffic from all namespaces or all pods

Here's what to look for in a good match:

```
apiVersion: networking.k8s.io/v1
```

```
kind: NetworkPolicy
```

```
metadata:
```

```
name: allow-frontend-to-backend
```

```
namespace: backend
```

```
spec:
```

```
podSelector:
```

```
matchLabels:
```

```
app: backend
```

```
ingress:
```

```
- from:
```

```
- namespaceSelector:
```

```
matchLabels:
```

name: frontend

Even better if the policy includes:

- namespaceSelector:

matchLabels:

name: frontend

podSelector:

matchLabels:

app: frontend

This limits access to pods in the frontend namespace with a specific label.

Step 5: Apply the correct NetworkPolicy

Once you've identified the best match, apply it:

```
kubectl apply -f ~/netpol/.yaml
```

Apply only one file. Do not alter or delete any existing sample.

```
ssh cka000056
```

```
kubectl get deployment -n frontend -o yaml
```

```
kubectl get deployment -n backend -o yaml
```

```
ls ~/netpol
```

```
cat ~/netpol/*.yaml # Review carefully
```

```
kubectl apply -f ~/netpol/.yaml
```

Command Summary

```
ssh cka000056
```

```
kubectl get deployment -n frontend -o yaml
```

```
kubectl get deployment -n backend -o yaml
```

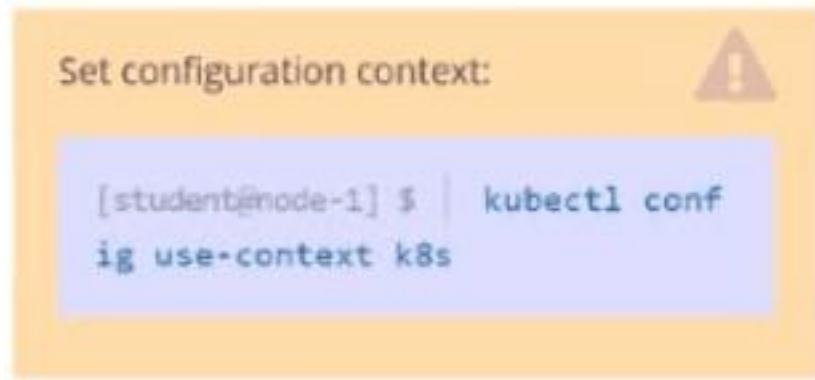
```
ls ~/netpol
```

```
cat ~/netpol/*.yaml # Review carefully
```

```
kubectl apply -f ~/netpol/.yaml
```

71. SIMULATION

Task Weight: 4%



Task

Schedule a Pod as follows:

- Name: kucc1
- App Containers: 2
- Container Name/Images: o nginx
o consul

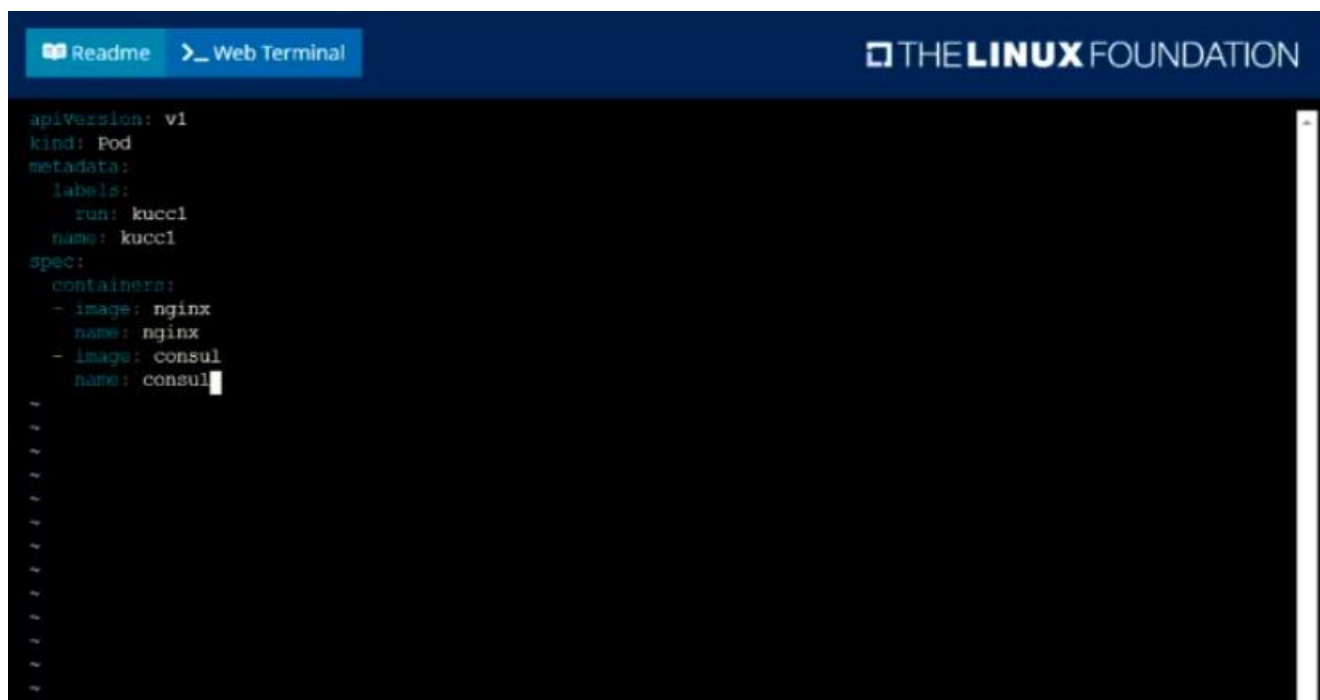
Answer:

See the solution below.

Explanation:

Solution:

```
student@node-1:~$ kubectl config use-context k8s
Switched to context "k8s".
student@node-1:~$ kubectl run kucc1 --image=nginx --dry-run=client -o yaml > aa.y
```





Graphical user interface, text, application

Description automatically generated

```
student@node-1:~$ kubectl config use-context k8s
Switched to context "k8s".
student@node-1:~$ kubectl run kucc1 --image=nginx --dry-run-client -o yaml > aa.yaml
student@node-1:~$ vim aa.yaml
student@node-1:~$ kubectl create -f aa.yaml
pod/kucc1 created
student@node-1:~$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
ll-factor-app        1/1     Running   0           6h34m
cpu-loader-98b9se    1/1     Running   0           6h33m
cpu-loader-ab2d3s    1/1     Running   0           6h33m
cpu-loader-kipb9a    1/1     Running   0           6h33m
foobar              1/1     Running   0           6h34m
front-end-6bc87b9748-24rcm 1/1     Running   0           5m4s
front-end-6bc87b9748-hd5wp 1/1     Running   0           5m2s
kucc1                0/2     ContainerCreating 0           3s
nginx-kusc00401      1/1     Running   0           2m28s
webserver-84c89dfd75-2dljn 1/1     Running   0           6h38m
webserver-84c89dfd75-8d8x2 1/1     Running   0           6h38m
webserver-84c89dfd75-z5zz4 1/1     Running   0           3m51s
student@node-1:~$
```

Text Description automatically generated

72. SIMULATION

For this item, you will have to ssh to the nodes ik8s-master-0 and ik8s-node-0 and complete all tasks on these nodes. Ensure that you return to the base node (hostname: node-1) when you have completed this item.

Context

As an administrator of a small development team, you have been asked to set up a Kubernetes cluster to test the viability of a new application.

Task

You must use kubeadm to perform this task. Any kubeadm invocations will require the use of the `--ignore-preflight-errors=all` option.

- ☞ Configure the node ik8s-master-0 as a master node. .
- ☞ Join the node ik8s-node-0 to the cluster.

Answer:

See the solution below.

Explanation:

solution

You must use the kubeadm configuration file located at `/etc/kubeadm.conf` when initializing your cluster.

You may use any CNI plugin to complete this task, but if you don't have your favourite CNI plugin's manifest



URL at hand, Calico is one popular option:

<https://docs.projectcalico.org/v3.14/manifests/calico.yaml>

Docker is already installed on both nodes and apt has been configured so that you can install the required tools.

73. SIMULATION

Create a pod as follows:

- ☞ Name: mongo
- ☞ Using Image: mongo
- ☞ In a new Kubernetes namespace named: my-website

Answer:

See the solution below.

Explanation:

solution

The screenshot shows a web terminal interface with a dark background. At the top, there are two tabs: 'Readme' and 'Web Terminal', and the 'THE LINUX FOUNDATION' logo on the right. The terminal shows the following commands and output:

```

root@node-1:~#
root@node-1:~#
root@node-1:~# k create ns my-website
namespace/my-website created
root@node-1:~# k run mongo --image=mongo -n my-website
pod/mongo created
root@node-1:~# k get po -n my-website

```

NAME	READY	STATUS	RESTARTS	AGE
mongo	0/1	ContainerCreating	0	4s

```

root@node-1:~#

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\9 B.JPG



74. SIMULATION

Get list of all pods in all namespaces and write it to file “/opt/pods-list.yaml”

Answer:

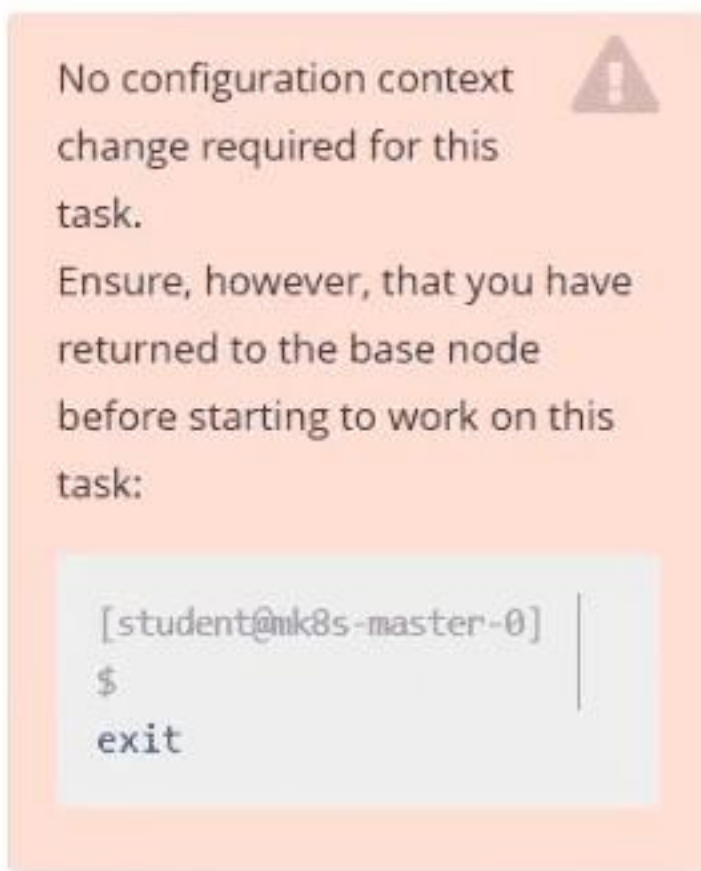
See the solution below.

Explanation:

```
kubectl get po --all-namespaces > /opt/pods-list.yaml
```

75. SIMULATION

Score: 7%



Task

First, create a snapshot of the existing etcd instance running at <https://127.0.0.1:2379>, saving the snapshot to /srv/data/etcd-snapshot.db.



Creating a snapshot of the given instance is expected to complete in seconds. If the operation seems to hang, something's likely wrong with your command. Use **CTRL + C** to cancel the operation and try again.

Next, restore an existing, previous snapshot located at `/var/lib/backup/etcd-snapshot-previous.db`

The following TLS certificates/key are supplied for connecting to the server with `etcdctl`:

- CA certificate:
`/opt/KUIN00601/ca.crt`
- Client certificate:
`/opt/KUIN00601/etcd-client.crt`
- Client key:
`/opt/KUIN00601/etcd-client.key`

Answer:

See the solution below.

Explanation:



Solution:

#backup

```
ETCDCTL_API=3 etcdctl --endpoints="https://127.0.0.1:2379" --
```

```
cacert=/opt/KUIN000601/ca.crt --cert=/opt/KUIN000601/etcd-client.crt --
```

```
key=/opt/KUIN000601/etcd-client.key snapshot save /etc/data/etcd-snapshot.db
```

#restore

```
ETCDCTL_API=3 etcdctl --endpoints="https://127.0.0.1:2379" --
```

```
cacert=/opt/KUIN000601/ca.crt --cert=/opt/KUIN000601/etcd-client.crt --
```

```
key=/opt/KUIN000601/etcd-client.key snapshot restore /var/lib/backup/etcd-snapshot- previoys.db
```

76. SIMULATION

Create a deployment as follows:

- ☞ Name: nginx-app
- ☞ Using container nginx with version 1.11.10-alpine
- ☞ The deployment should contain 3 replicas

Next, deploy the application with new version 1.11.13-alpine, by performing a rolling update.

Finally, rollback that update to the previous version 1.11.10-alpine.

Answer:

See the solution below.

Explanation:

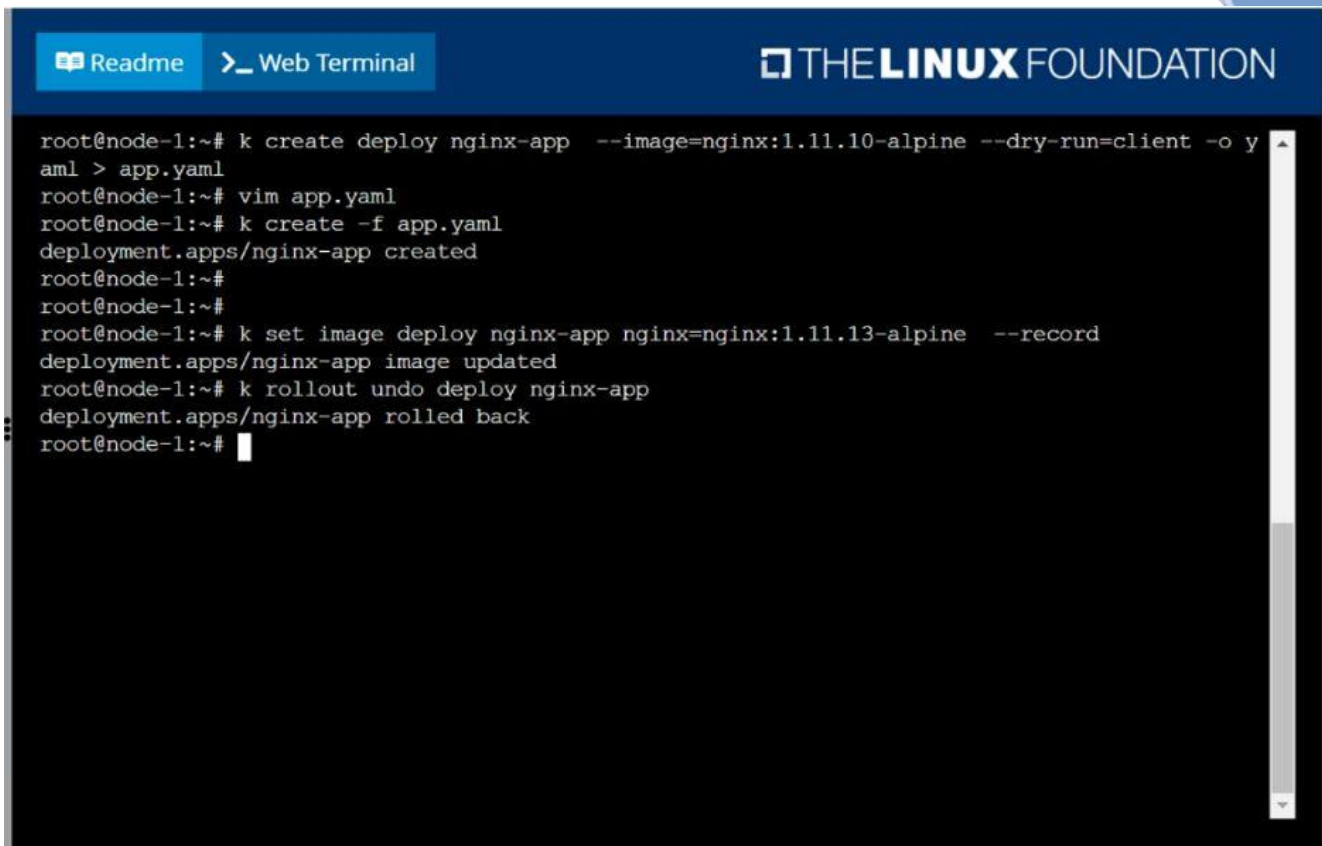
solution

A screenshot of a web terminal interface. The top header is dark blue with two buttons on the left: 'Readme' (with a book icon) and 'Web Terminal' (with a terminal icon). On the right side of the header is 'THE LINUX FOUNDATION' logo. The terminal window has a black background with white text. The prompt is 'root@node-1:~#'. The first command entered is 'k create deploy nginx-app --image=nginx:1.11.10-alpine --dry-run=client -o y', which is split across two lines. The second line shows 'aml > app.yaml'. The third line shows the prompt 'root@node-1:~#' followed by 'vim app.yaml' and a white cursor. A vertical scrollbar is visible on the right side of the terminal window.

F:\Work\Data Entry Work\Data Entry\20200827\CKA\7 B.JPG

A screenshot of a terminal window with a dark blue header bar. The header contains three elements: a "Readme" button with a book icon, a "Web Terminal" button with a cursor icon, and "THE LINUX FOUNDATION" logo. The main area of the terminal displays a YAML configuration for a Deployment named "nginx-app". The configuration includes fields for apiVersion, kind, metadata, spec (with replicas set to 3), and template (with container image nginx:1.11.10-alpine). On the left side of the terminal, there are vertical ellipsis and tilde symbols indicating file navigation. At the bottom left, the filename "app.yaml" is shown. A vertical scrollbar is visible on the right edge.

F:\Work\Data Entry Work\Data Entry\20200827\CKA\7 C.JPG



```

root@node-1:~# k create deploy nginx-app --image=nginx:1.11.10-alpine --dry-run=client -o y
aml > app.yaml
root@node-1:~# vim app.yaml
root@node-1:~# k create -f app.yaml
deployment.apps/nginx-app created
root@node-1:~#
root@node-1:~#
root@node-1:~# k set image deploy nginx-app nginx=nginx:1.11.13-alpine --record
deployment.apps/nginx-app image updated
root@node-1:~# k rollout undo deploy nginx-app
deployment.apps/nginx-app rolled back
root@node-1:~#

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\7 D.JPG

77. SIMULATION

Create a deployment as follows:

- ☞ Name: nginx-random
- ☞ Exposed via a service nginx-random
- ☞ Ensure that the service and pod are accessible via their respective DNS records
- ☞ The container(s) within any pod(s) running as a part of this deployment should use the nginx Image

Next, use the utility nslookup to look up the DNS records of the service and pod and write the output to /opt/KUNW00601/service.dns and /opt/KUNW00601/pod.dns respectively.

Answer:

See the solution below.

Explanation:

Solution:



ATION

Readme
Web Terminal

```

root@node-1:~# k create deploy nginx-random --image=nginx
deployment.apps/nginx-random created
root@node-1:~# k expose deploy nginx-random --name=nginx-random --port=80 --target-port=80
service/nginx-random exposed
root@node-1:~# vim dns.yaml
root@node-1:~# k create -f dns.yaml
pod/busybox1 created
root@node-1:~# k get po -o wide | grep nginx-random
nginx-random-6d5766bbdc-ptzv2 1/1 Running 0 103s 10.244.2.16 k8s-node-1 <none> <none>
root@node-1:~# k exec -it busybox1 -- nslookup nginx-random
Server: 10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name: nginx-random
Address 1: 10.111.37.132 nginx-random.default.svc.cluster.local
root@node-1:~# k exec -it busybox1 -- nslookup nginx-random > /opt/KUNW00601/service.dns
root@node-1:~# k exec -it busybox1 -- nslookup 10-244-2-16.default.pod
Server: 10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name: 10-244-2-16.default.pod
Address 1: 10.244.2.16 10-244-2-16.nginx-random.default.svc.cluster.local
root@node-1:~# k exec -it busybox1 -- nslookup 10-244-2-16.default.pod > /opt/KUNW00601/pod.dns

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\17 E.JPG

78. SIMULATION

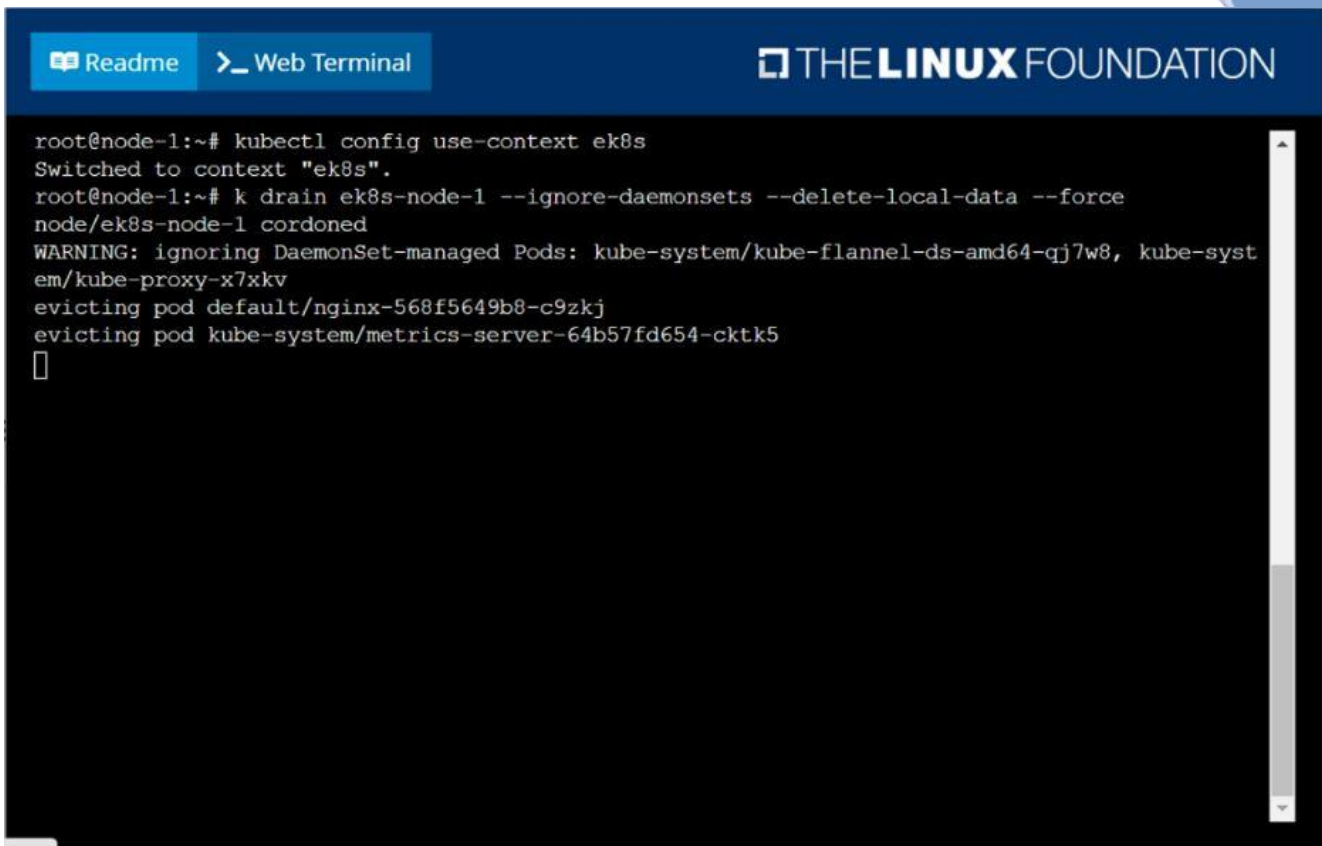
Set the node named ek8s-node-1 as unavailable and reschedule all the pods running on it.

Answer:

See the solution below.

Explanation:

solution



```

root@node-1:~# kubectl config use-context ek8s
Switched to context "ek8s".
root@node-1:~# k drain ek8s-node-1 --ignore-daemonsets --delete-local-data --force
node/ek8s-node-1 cordoned
WARNING: ignoring DaemonSet-managed Pods: kube-system/kube-flannel-ds-amd64-qj7w8, kube-system/kube-proxy-x7xkv
evicting pod default/nginx-568f5649b8-c9zkj
evicting pod kube-system/metrics-server-64b57fd654-cktk5
[]
    
```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\19 B.JPG

79. SIMULATION

Create an nginx pod and list the pod with different levels of verbosity

Answer:

See the solution below.

Explanation:

// create a pod

```
kubectl run nginx --image=nginx --restart=Never --port=80
```

// List the pod with different verbosity

```
kubectl get po nginx --v=7
```

```
kubectl get po nginx --v=8
```

```
kubectl get po nginx --v=9
```

80. SIMULATION

Create a pod that having 3 containers in it? (Multi-Container)



Answer:

See the solution below.

Explanation:

image=nginx, image=redis, image=consul

Name nginx container as "nginx-container"

Name redis container as "redis-container"

Name consul container as "consul-container"

Create a pod manifest file for a container and append container

section for rest of the images

kubectrl run multi-container --generator=run-pod/v1 --image=nginx --

dry-run -o yaml > multi-container.yaml

then

vim multi-container.yaml

apiVersion: v1

kind: Pod

metadata:

labels:

run: multi-container

name: multi-container

spec:

containers:

- image: nginx

name: nginx-container

- image: redis

name: redis-container

- image: consul

name: consul-container

restartPolicy: Always

81. SIMULATION



Create a persistent volume with name app-data, of capacity 2Gi and access mode ReadWriteMany. The type of volume is hostPath and its location is /srv/app-data.

Answer:

See the solution below.

Explanation:

solution

Persistent Volume

A persistent volume is a piece of storage in a Kubernetes cluster. PersistentVolumes are a cluster-level resource like nodes, which don't belong to any namespace. It is provisioned by the administrator and has a particular file size. This way, a developer deploying their app on Kubernetes need not know the underlying infrastructure. When the developer needs a certain amount of persistent storage for their application, the system administrator configures the cluster so that they consume the PersistentVolume provisioned in an easy way.

Creating Persistent Volume

```
kind: PersistentVolume
apiVersion: v1
metadata:
  name: app-data
spec:
  capacity: # defines the capacity of PV
  we are creating storage: 2Gi #the amount of storage we are trying to claim
  accessModes: # defines the
  rights of the volume we are creating - ReadWriteMany
  hostPath:
    path: "/srv/app-data" # path to which we
  are creating the volume
```

Challenge

👁️ Create a Persistent Volume named app-data, with access mode ReadWriteMany, storage classname shared, 2Gi of storage capacity and the host path /srv/app- data.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: app-data
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: /srv/app-data
  storageClassName: shared
```

* 2. Save the file and create the persistent volume.

```
njerry191@cloudshell:~ (extreme-clone-265411)$ kubectl create -f pv.yaml
persistentvolume/pv created
```

Image for post

* 3. View the persistent volume.

```
njerry191@cloudshell:~ (extreme-clone-265411)$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	REASON	AGE
app-data	2Gi	RWX	Retain	Available		shared		31s

☞ Our persistent volume status is available meaning it is available and it has not been mounted yet. This status will change when we mount the persistentVolume to a persistentVolumeClaim.

PersistentVolumeClaim

In a real ecosystem, a system admin will create the PersistentVolume then a developer will create a PersistentVolumeClaim which will be referenced in a pod. A PersistentVolumeClaim is created by specifying the minimum size and the access mode they require from the persistentVolume. Challenge

☞ Create a Persistent Volume Claim that requests the Persistent Volume we had created above. The claim should request 2Gi. Ensure that the Persistent Volume Claim has the same storageClassName as the persistentVolume you had previously created.

```
kind: PersistentVolumeapiVersion: v1metadata: name:app-data
```

spec:

accessModes: - ReadWriteMany resources:

```
requests: storage: 2Gi
```



storageClassName: shared

* 2. Save and create the pvc

njerry191@cloudshell:~ (extreme-clone-2654111)\$ kubectl create -f app-data.yaml

persistentvolumeclaim/app-data created

* 3. View the pvc

```
njerry191@cloudshell:~ (extreme-clone-2654111)$ kubectl get pvc
NAME      STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
pv        Bound     pv        512m       RWX             shared
```

Image for post

* 4. Let's see what has changed in the pv we had initially created.

```
njerry191@cloudshell:~ (extreme-clone-2654111)$ kubectl get pv
NAME      CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM          STORAGECLASS   REASON   AGE
pv        512m       RWX             Retain            Bound    default/pv     shared         16m
```

Image for post

Our status has now changed from available to bound.

* 5. Create a new pod named myapp with image nginx that will be used to Mount the Persistent Volume

Claim with the path /var/app/config.

Mounting a Claim

apiVersion: v1kind: Podmetadata: creationTimestamp: null name: app-dataspec: volumes: -

name:configpvc persistenVolumeClaim: claimName: app-data containers: - image: nginx name: app

volumeMounts: - mountPath: "/srv/app-data " name: configpvc

82. SIMULATION

Create a pod as follows:

- ☞ Name: non-persistent-redis
- ☞ container Image: redis
- ☞ Volume with name: cache-control
- ☞ Mount path: /data/redis

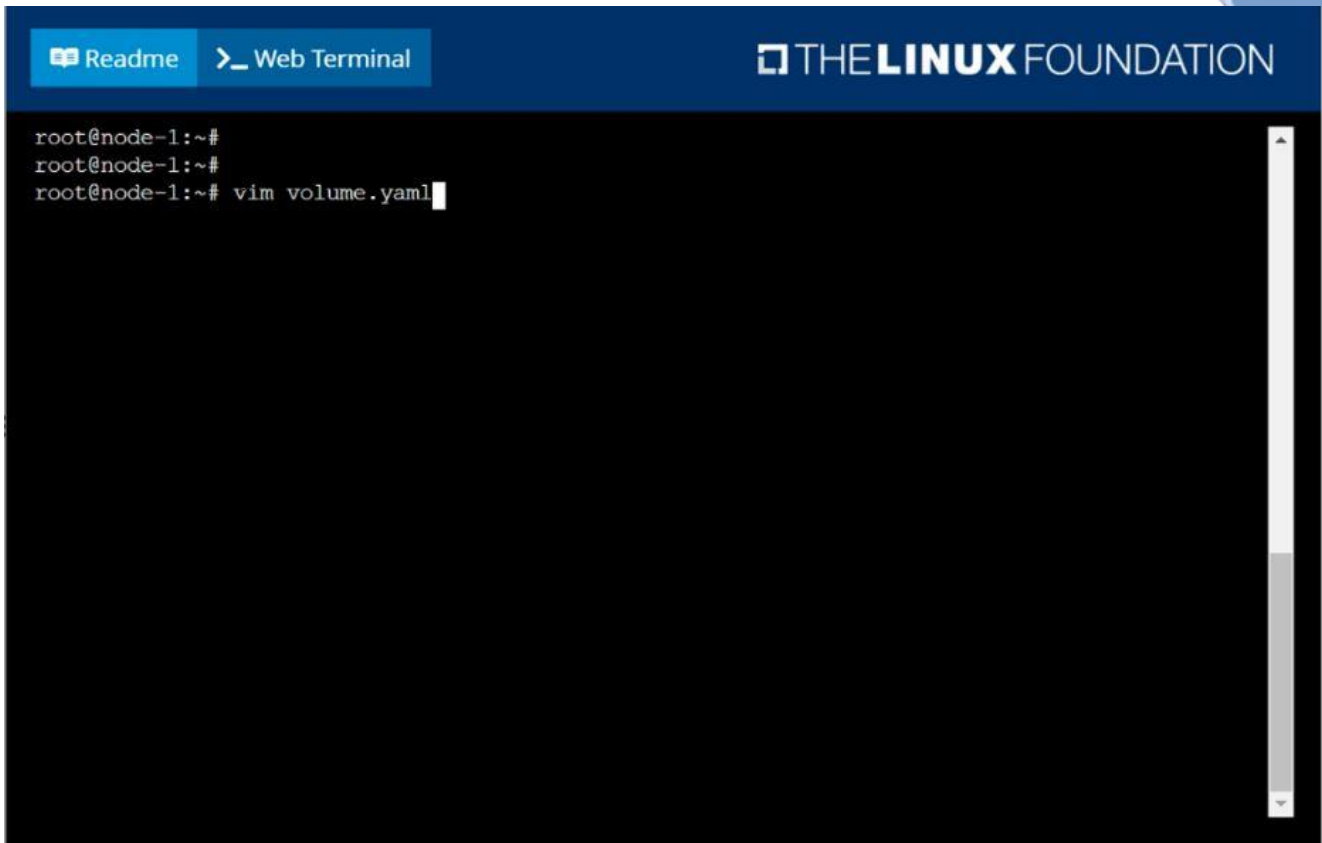
The pod should launch in the staging namespace and the volume must not be persistent.

Answer:

See the solution below.

Explanation:

solution

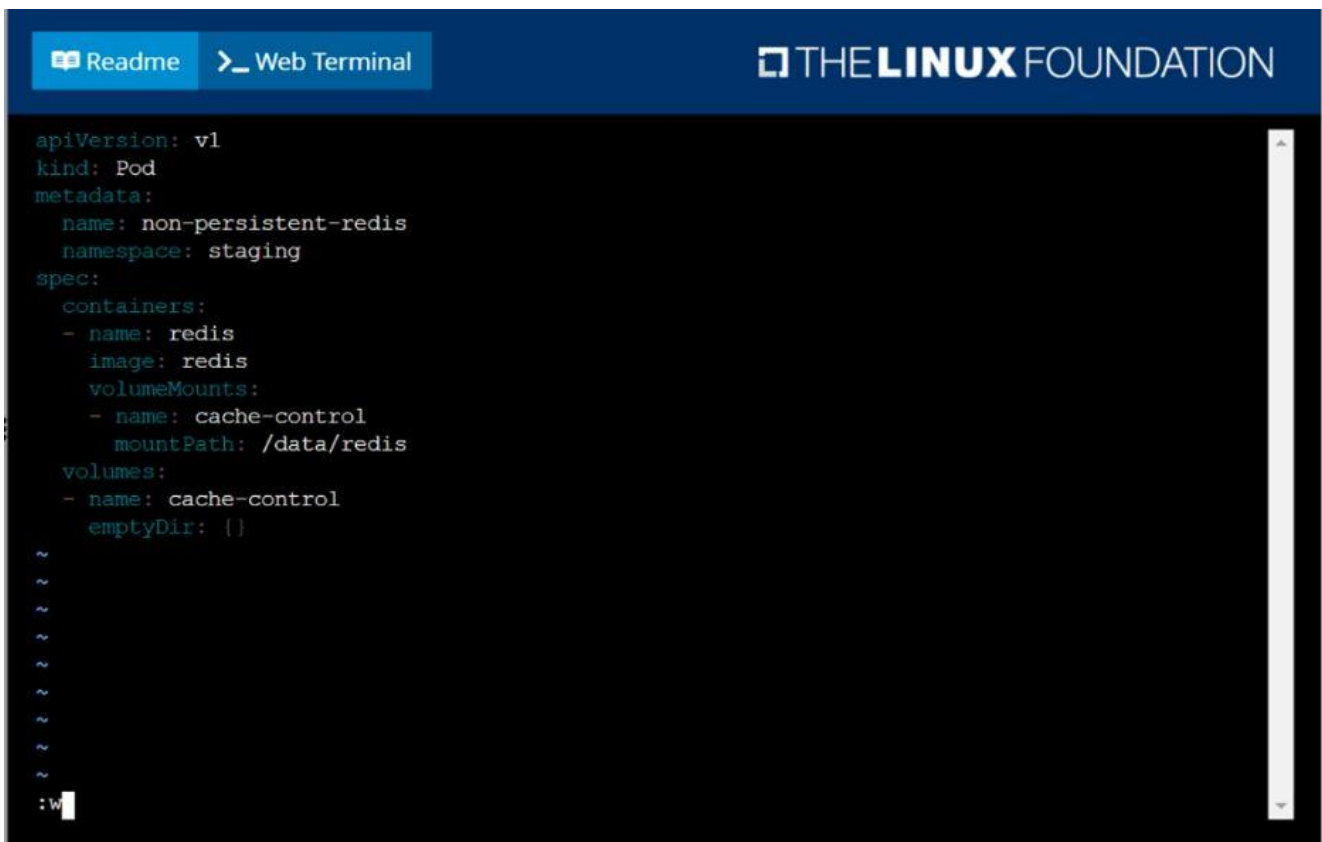


The screenshot shows a web terminal window with a dark background. At the top, there is a blue header bar with a 'Readme' button and a 'Web Terminal' button. The terminal text shows a user at the root of a node-1 machine, running the command 'vim volume.yaml'.

```

root@node-1:~#
root@node-1:~#
root@node-1:~# vim volume.yaml
  
```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\13 B.JPG

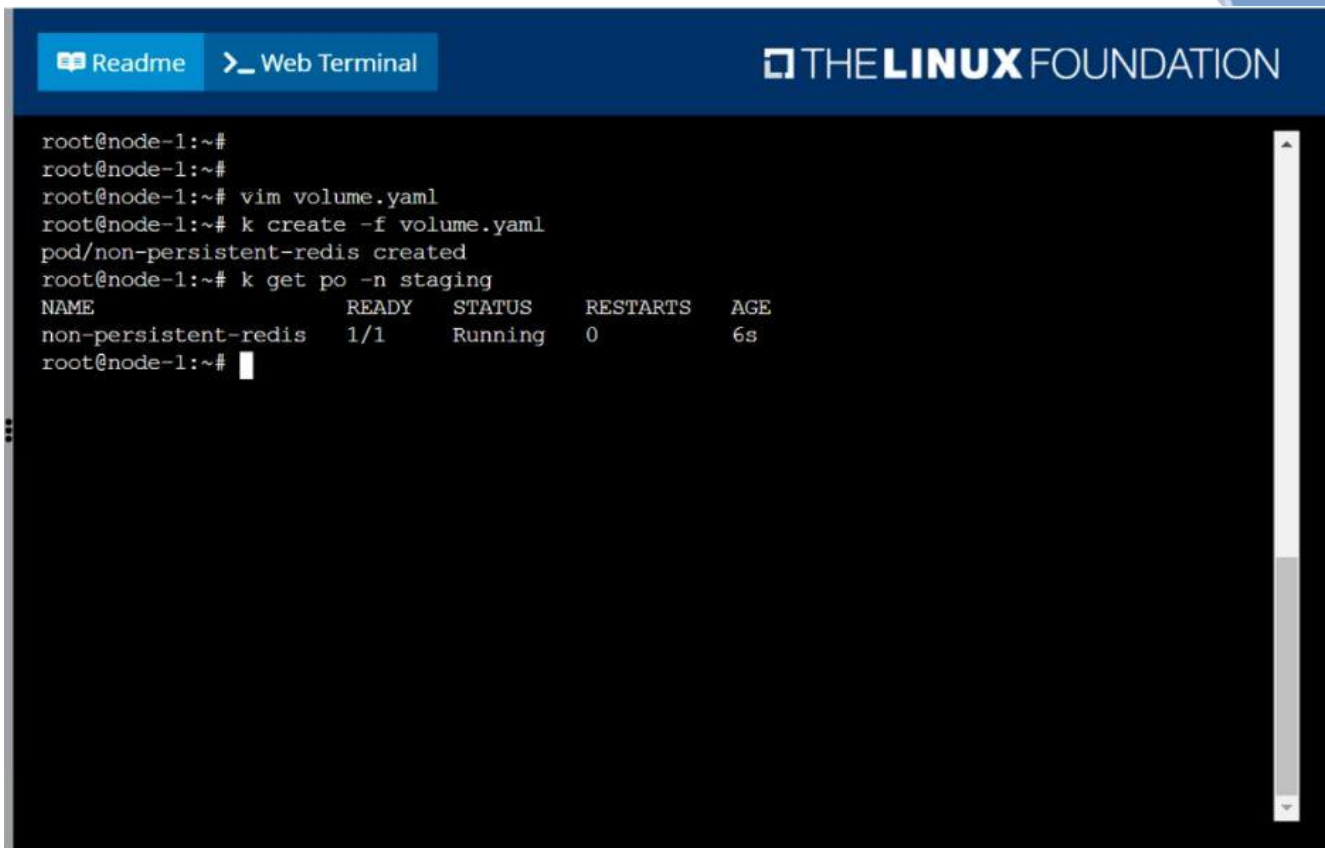


The screenshot shows a web terminal window with a dark background. At the top, there is a blue header bar with a 'Readme' button and a 'Web Terminal' button. The terminal text shows the contents of a Kubernetes manifest file named 'volume.yaml'.

```

apiVersion: v1
kind: Pod
metadata:
  name: non-persistent-redis
  namespace: staging
spec:
  containers:
  - name: redis
    image: redis
    volumeMounts:
    - name: cache-control
      mountPath: /data/redis
  volumes:
  - name: cache-control
    emptyDir: {}
~
~
~
~
~
~
~
~
~
~
:w
  
```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\13 C.JPG



```

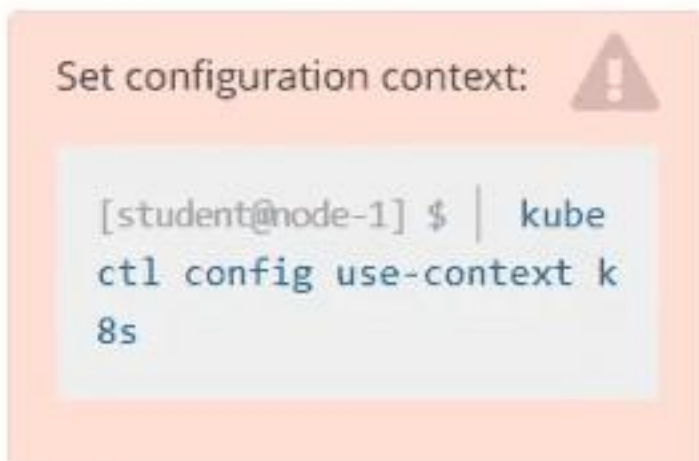
root@node-1:~#
root@node-1:~#
root@node-1:~# vim volume.yaml
root@node-1:~# k create -f volume.yaml
pod/non-persistent-redis created
root@node-1:~# k get po -n staging
NAME                READY   STATUS    RESTARTS   AGE
non-persistent-redis 1/1     Running   0           6s
root@node-1:~#

```

F:\Work\Data Entry Work\Data Entry\20200827\CKA\13 D.JPG

83. SIMULATION

Score: 4%



Task

Create a pod named kucc8 with a single app container for each of the following images running inside (there may be between 1 and 4 images specified): nginx + redis + memcached .

Answer:



See the solution below.

Explanation:

Solution:

```
kubectrl run kucc8 --image=nginx --dry-run -o yaml > kucc8.yaml
```

```
# vi kucc8.yaml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
creationTimestamp: null
```

```
name: kucc8
```

```
spec:
```

```
containers:
```

```
- image: nginx
```

```
name: nginx
```

```
- image: redis
```

```
name: redis
```

```
- image: memcached
```

```
name: memcached
```

```
- image: consul
```

```
name: consul
```

```
#
```

```
kubectrl create -f kucc8.yaml
```

```
#12.07
```